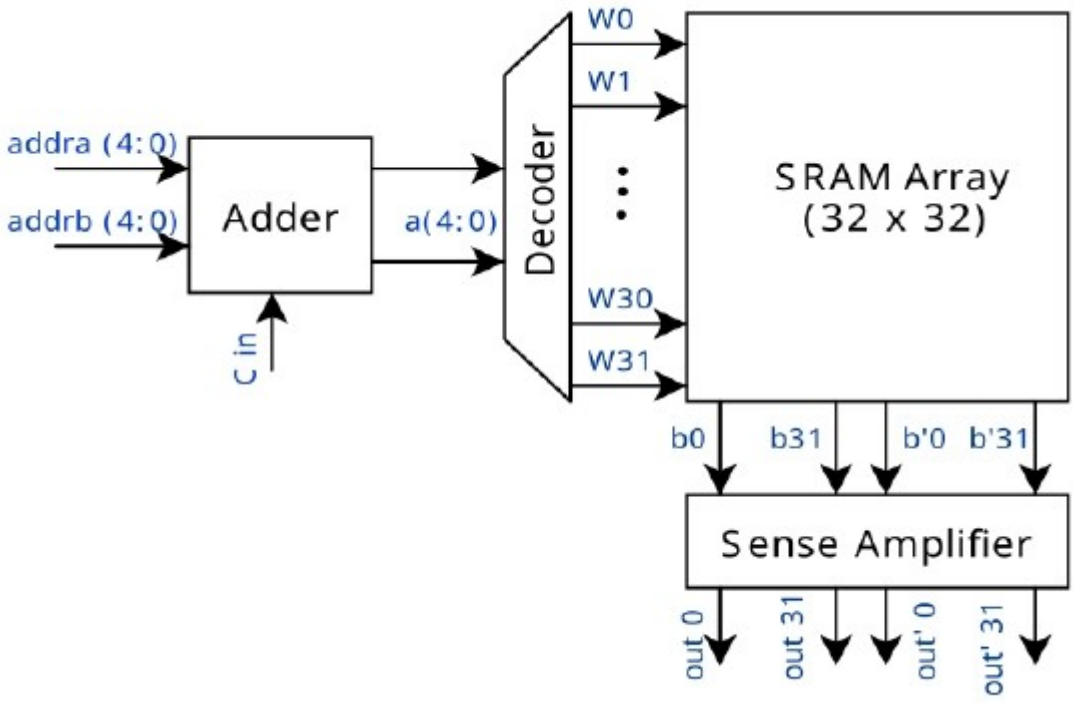
This page is visible to anyone with the link.

DVD PROJECT : 32 x 32-bit SRAM

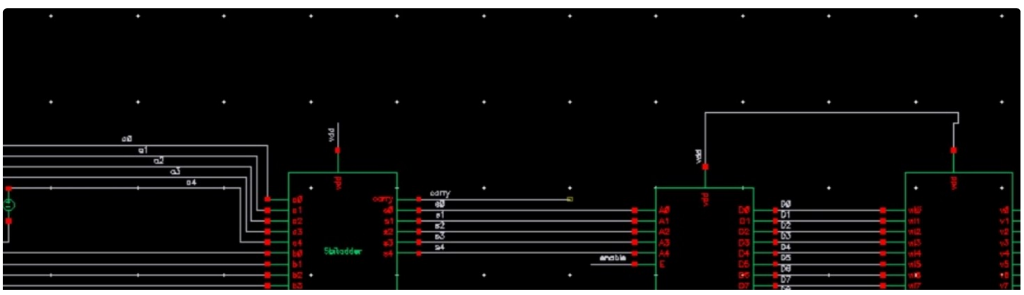
Team

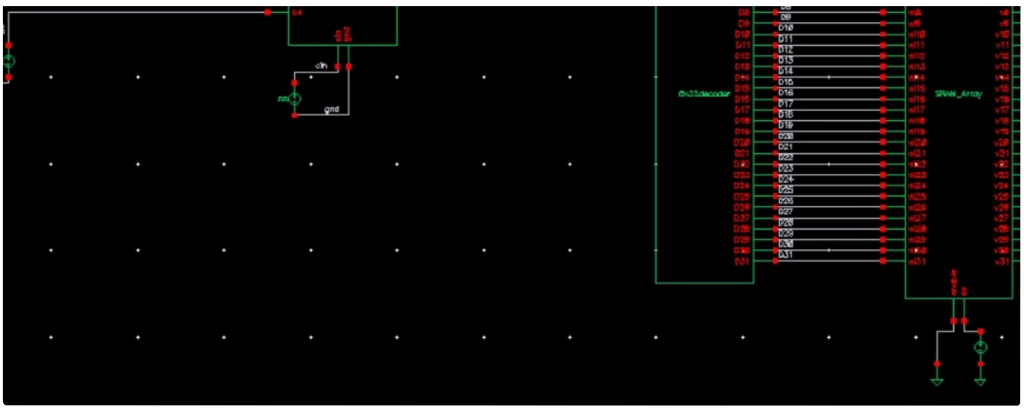
- Pragnya (2023102067)
  Aikya (2023102062)
- Indira (2023102070)
  Abhinaya (2023102071)

We were assigned the TSMC 28 Node



Our Circuit:





1. Introduction

Static Random Access Memory (SRAM) is a key building block in almost all digital systems.

In this project, we designed and analyzed a **32 × 32-bit SRAM** system starting from the **6T SRAM cell** and building up to the **array level**, with supporting **adder, decoder, and sense amplifier** blocks.

Primary goals:

- To design a robust 6T SRAM cell in the 28 nm TSMC process.
- To characterize **read and write noise margins** and verify their stability.
- To integrate the cell into a **32 × 32 array** accessed using a 5-bit word address.
- To design the **5-bit adder**, **5-to-32 decoder**, and **sense amplifiers**, and to verify their **functionality**, **latency**, and basic **power behavior**.
- And to apply **design optimizations** for lower delay and improved efficiency.

2. Block - Level

2.1 Overall Architecture

The top-level SRAM system consists of 4 main blocks:

1. **Adder**

- Inputs: two 5-bit addresses `a[4:0]`, `b[4:0]`, and carry-in `cin`.
- Output: 5-bit summed address `s[4:0]`.
- Function: Computes the effective memory address used to access the SRAM array (supports FIFO-like addressing).

2. **5-to-32 Decoder**

- Input: `s[4:0]` and enable signal `enable` .
- Outputs: selects one of the word lines `wl0–wl31` .
- Function: Activates exactly **one** row of the 32-word SRAM array. `enable = 0` forces all word lines LOW, allowing bit-line pre-charge.

3. **32 × 32 SRAM Array**

- Contains **32 words**, each word containing **32 identical 6T SRAM cells**.
- Each column connects to differential bit lines `bl[i]` and `blb[i]` for (i = 0:31).
- Word line `wl[i]` connects all cells in row *i* simultaneously to the bit lines.

4. **Sense Amplifier Array**

- Each bit-line pair `(bl[i], blb[i])` is connected to a **differential sense amplifier**.
- Outputs: `v0–v31` (full-swing digital outputs).
- Function: Detects the small voltage difference developed on the bit-line pair during a read and amplifies it to rail-to-rail logic levels.

Overall operation:

- The adder generates the effective 5-bit address.
- The decoder asserts the selected word line.
- The corresponding row of 6T cells perturbs the precharged bit lines.
- Sense amplifiers resolve and output the 32-bit data word.

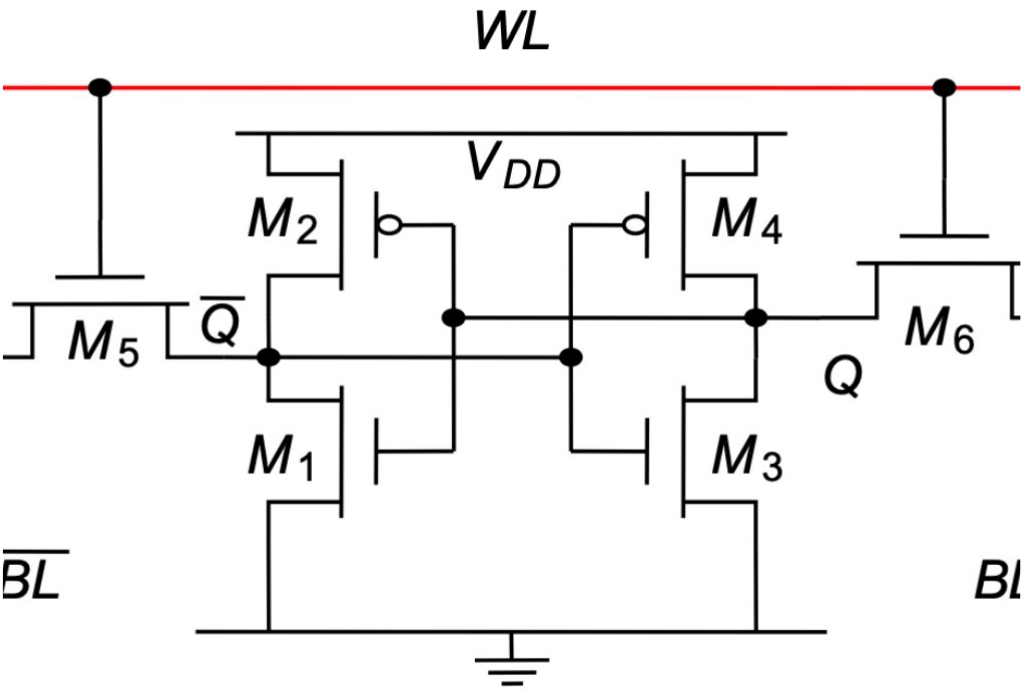
3. 6T SRAM Cell Design and Analysis

6-transistor (6T) SRAM cell is the fundamental storage element in static RAM.

It stores a single bit of data (“0” or “1”) using two cross-coupled CMOS inverters, which maintain a stable bistable state as long as power is supplied.

Each cell can be **read** or **written** through two **access transistors**, controlled by the **word line (WL)** and connected to the **bit lines (BL,**

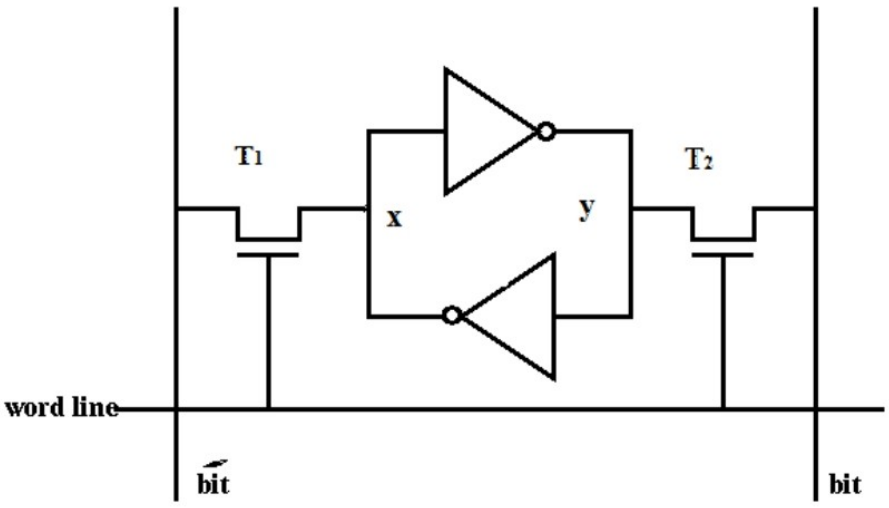
BLB).



3.1 Cell Structure

The **6T SRAM cell** consists of:

- **Cross-coupled inverters**
  - INV1: M1 (pull-down nMOS), M2 (pull-up pMOS)
  - INV2: M3 (pull-down nMOS), M4 (pull-up pMOS)



Each inverter consists of a **pull-up pMOS** and a **pull-down nMOS** transistor:

- **Access transistors**
  - M5 and M6, nMOS, controlled by the **word line (WL)**
  - M5 connects node (Q) to bit line BL
  - M6 connects node ($\bar{Q}$) to bit line BLB

The cross-coupled inverters form a **bistable latch**:

- State 1: (Q = 1, $\bar{Q}$ = 0)
- State 0: (Q = 0, $\bar{Q}$ = 1)

As long as VDD is applied, the cell remains in one of these two stable states.

3.2 Components

- **Word Line (WL)**
  - Controls M5 and M6.
  - WL = 1: access transistors ON, cell connected to bit lines (read/write enabled).
  - WL = 0: access transistors OFF, cell isolated (hold mode).
- **Bit Lines (BL, BLB)**
  - Differential pair used for both read and write.
  - During read: both precharged high; cell discharges one line slightly.
  - During write: one line driven LOW and the other HIGH by the write driver.
- **Access Transistors (M5, M6)**
  - Interface between storage nodes and bit lines.
  - Their strength strongly affects **write-ability** and **read stability**.
- **Cross-Coupled Inverters (M1–M4)**
  - Provide positive feedback to store the bit.
  - Relative sizing of pull-up, pull-down, and access devices determines **cell ratio**, **noise margins**, and **speed**.

3.3 Transistor Sizing (28 nm TSMC)

Chosen sizes (28 nm process):

- Access transistors (M5, M6):

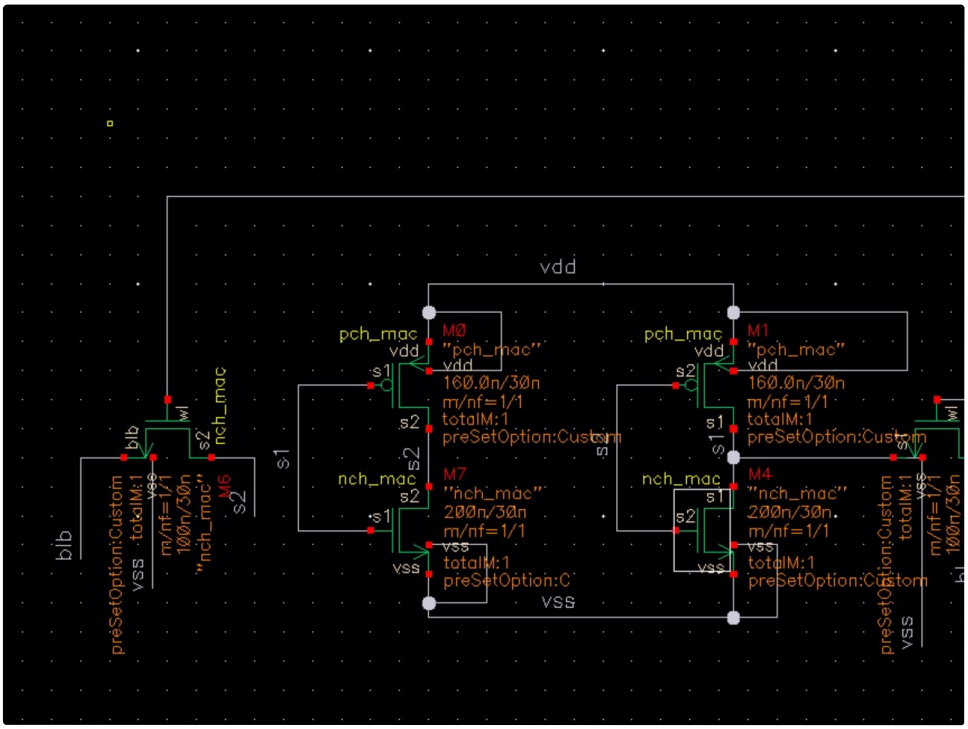
`W = 100nm; L = 30nm`

- Pull-down nMOS (M1, M3):

  `W = 250nm; L = 30nm`

- Pull-up pMOS (M2, M4):

  `W = 160nm; L = 30nm`



Key ratios (first-order)

1. **Cell ratio (CR)** – pull-down versus access:

$$CR = \frac{I_{pd}}{I_a} = \frac{\beta_{pd}}{\beta_a} = \frac{W/L_{pd}}{W/L_a} = \frac{160/30}{100/30} \approx 1.6$$

2. **Write-ability ratio** – access versus pull-up:

$$\frac{I_a}{I_{pu}} = \frac{\mu_n (W/L)_a}{\mu_p (W/L)_{\text{pu}}} \approx 1.25$$

These choices **balance read stability and write-ability**:

- Larger pull-downs (M1, M3) help avoid read disturb.
- Sufficiently strong access devices (M5, M6) allow the write drivers to flip the cell.
- Pull-ups are weaker to assist writing, but still strong enough for hold stability.

4. Operating Modes of the 6T Cell

4.1 Hold (Idle) Mode

- WL = 0 → M5, M6 OFF.
- Nodes (Q) and ($\bar{Q}$) are only connected via the cross-coupled inverters.
- There is no DC path to BL or BLB, so the stored data is retained indefinitely.
- The stored data is retained as long as VDD is present.

4.2 Read Operation

1. **Precharge**
   - BL and BLB are precharged to VDD before WL is activated.
2. **Word Line Activation**
   - WL = 1 turns ON the access transistors.
   - (Q) connects to BL and ($\bar{Q}$) connects to BLB.
3. **Differential Development**
   - If the cell stores 1 ((Q = 1, $\bar{Q}$ = 0)):
     - BL stays close to VDD.
     - BLB discharges slightly through M6 and the pull-down.
   - If the cell stores 0, the opposite happens:
     - BL discharges slightly; BLB stays near VDD.
4. **Sensing and Restore**
   - The sense amplifier detects which line is slightly lower.
   - Since the difference is small , differential sensing allows **high-speed and low-power** read.
   - After sensing, WL goes low again and the precharge circuitry returns BL and BLB to VDD.

The sizing ensures that the voltage rise on the “0” storage node during read is small, so the cell does not unintentionally flip.

4.3 Write Operation

1. **Bit-line Setup**
    - To write ‘1’: BL = 1, BLB = 0.
    - To write ‘0’: BL = 0, BLB = 1.
2. **Word Line Activation**
    - WL = 1 connects bit lines to the storage nodes.
3. **Overpower and Flip**
    - The node connected to the LOW bit line begins to discharge.
    - Positive feedback in the cross-coupled inverters accelerates the transition, latching the new state.
4. **Completion**
    - WL is lowered and the cell is isolated again.
    - Bit lines return to the precharged state.

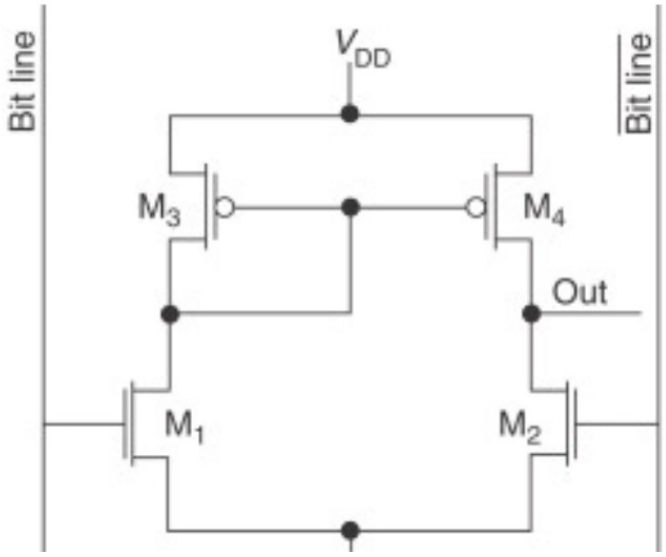
4.4 Precharging

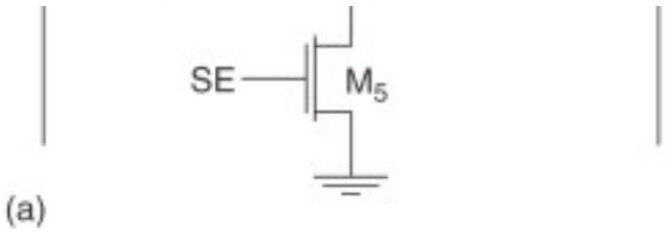
Before every **read or write**, both **bit lines (BL and ~BL)** are brought to a **known voltage level**.

Precharge sets both Bitlines to a known voltage (usually Vdd) so the sense amplifier can reliably detect the tiny voltage difference during a Read. It also ensures Bitlines can swing fully during Writes and prevents floating voltages when idle.

4.5 Sense Amplifier

The Sense Amplifier is a **small, high-speed differential amplifier** used during the **read operation** in SRAM.





(a)

Its used to recognize which bit is stored in the READ operation.

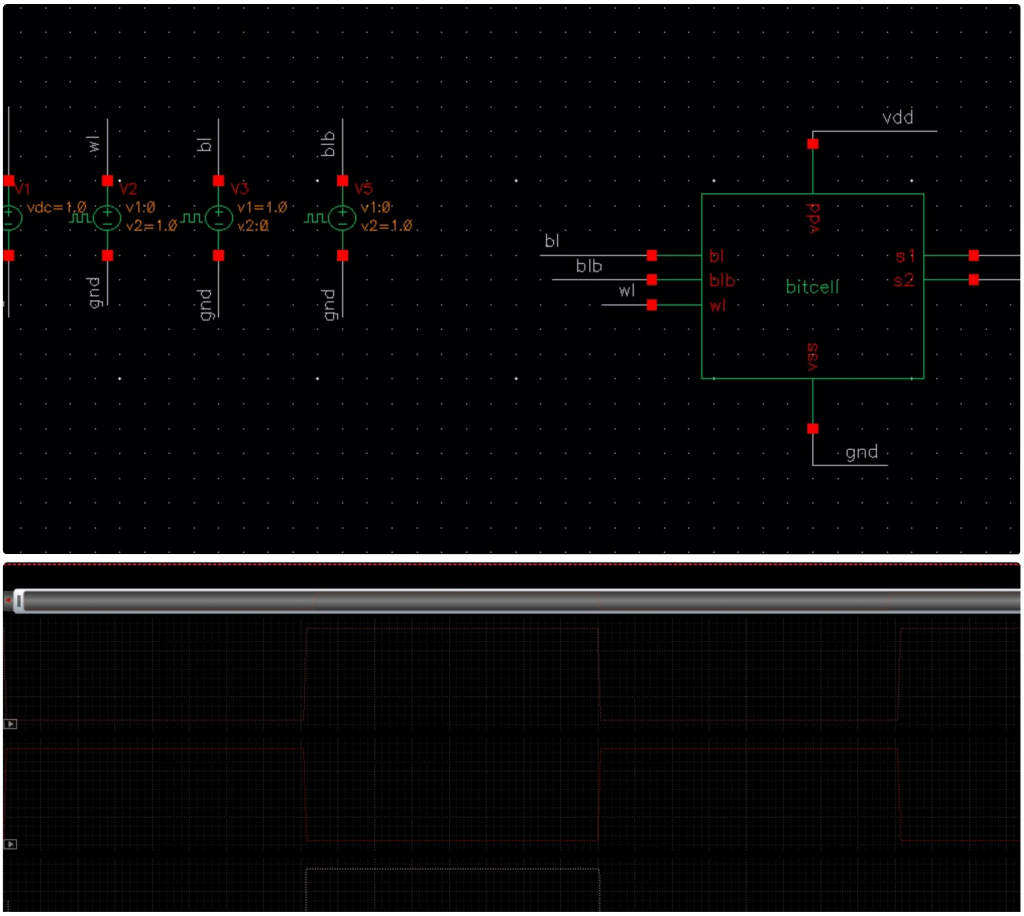
When a voltage difference develops across its terminals, it amplifies it and based on whether it is positive or negative it can quickly identify if 0 or 1 was stored.

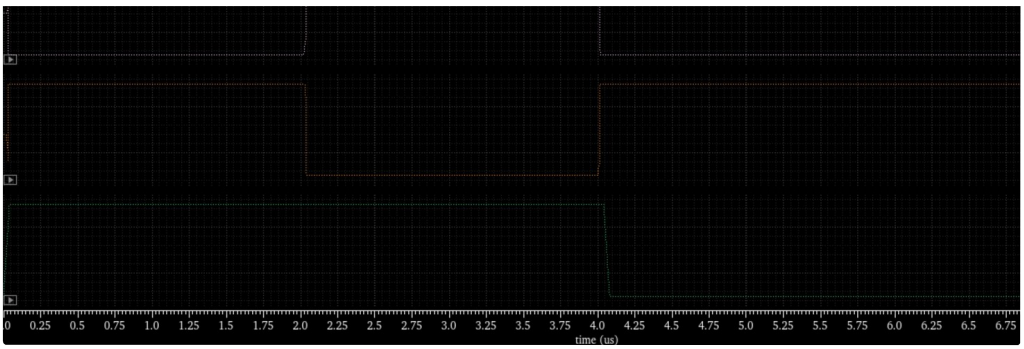
- **0 stored:** BL → discharges slightly → SA detects BL < ~BL → outputs 0
- **1 stored:** ~BL → discharges slightly → SA detects BL > ~BL → outputs 1

It mainly used because -

- Without differential sensing, detecting small voltage swings would be too slow.
- Positive feedback in the SA ensures **full rail-to-rail swing**.

Schematic and Plot:

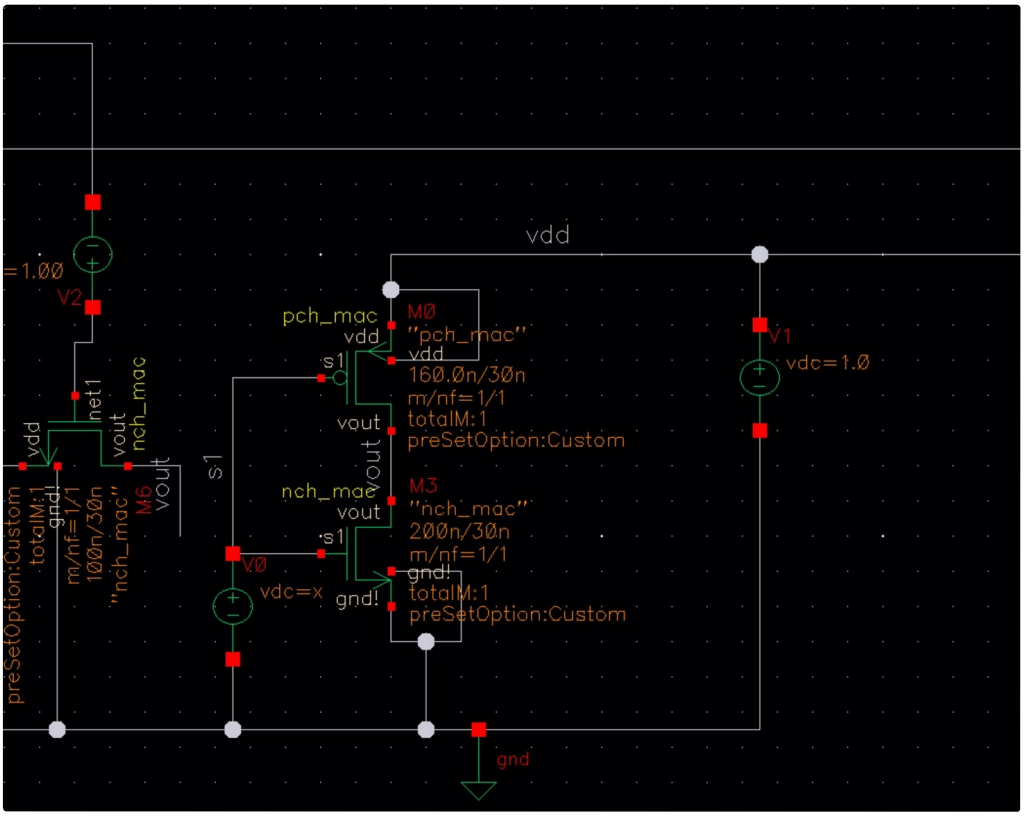


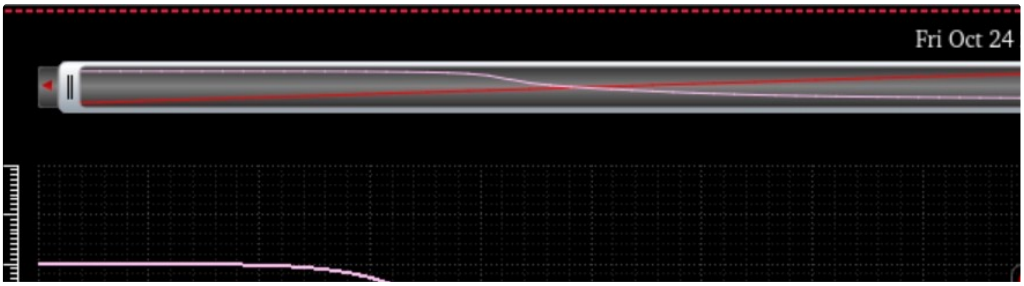


The plots show the **transient voltage responses** of the word line (WL), bit lines (BL and $\overline{BL}$), and the output node (Q) during **read and write operations** for the chosen W/L sizing in the 28 nm TSMC process.

5. Noise Margin Characterization

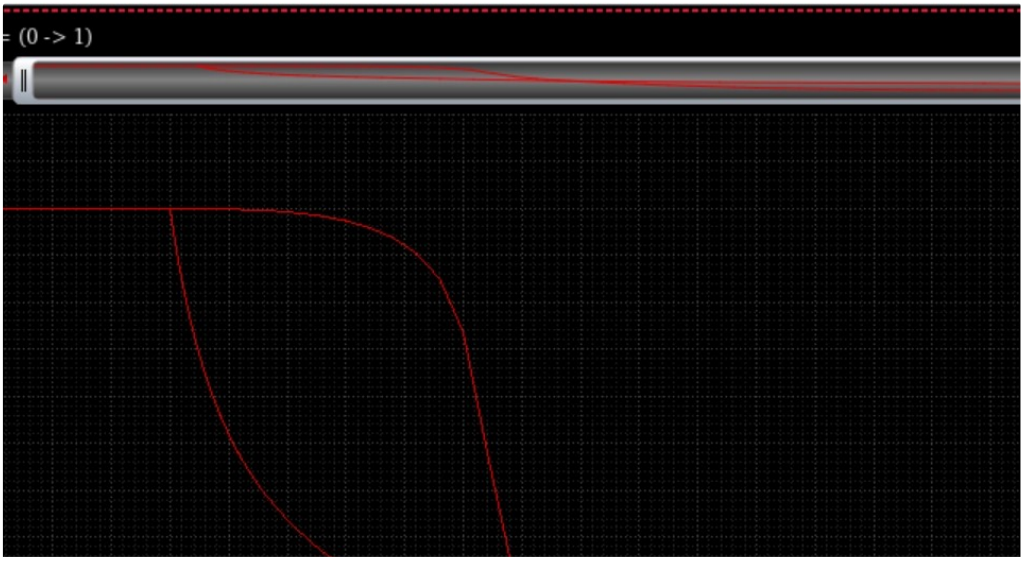
5.1 Read Static Noise Margin (SNM)

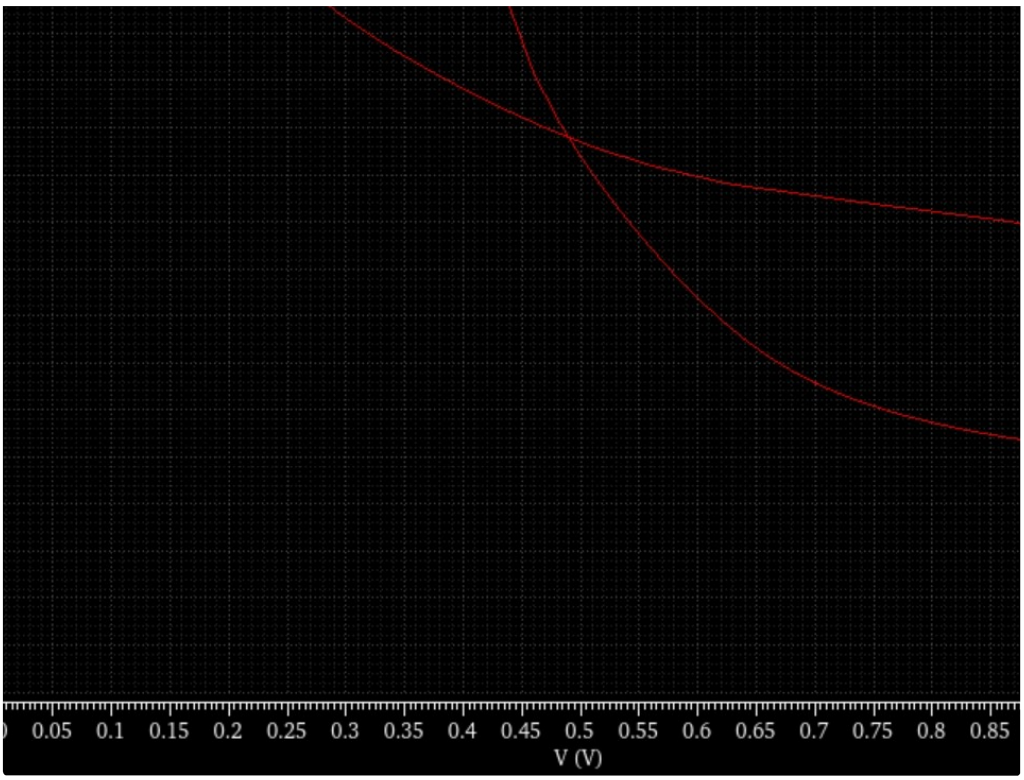






This plot shows the **voltage transfer characteristics (VTC)** of the two cross-coupled inverters that form the 6T SRAM cell. One inverter's VTC is mirrored and overlapped with the other to form a **butterfly-shaped curve**.





Method

- Sweep the input node, (V_1), from 0 to VDD and record the output, (V_2), to obtain the inverter VTC.
- Mirror this VTC about the line, $(V_1 = V_2)$, to generate the **butterfly plot**.
- The **read SNM** is the side of the largest square that fits between the two curves.

Observations

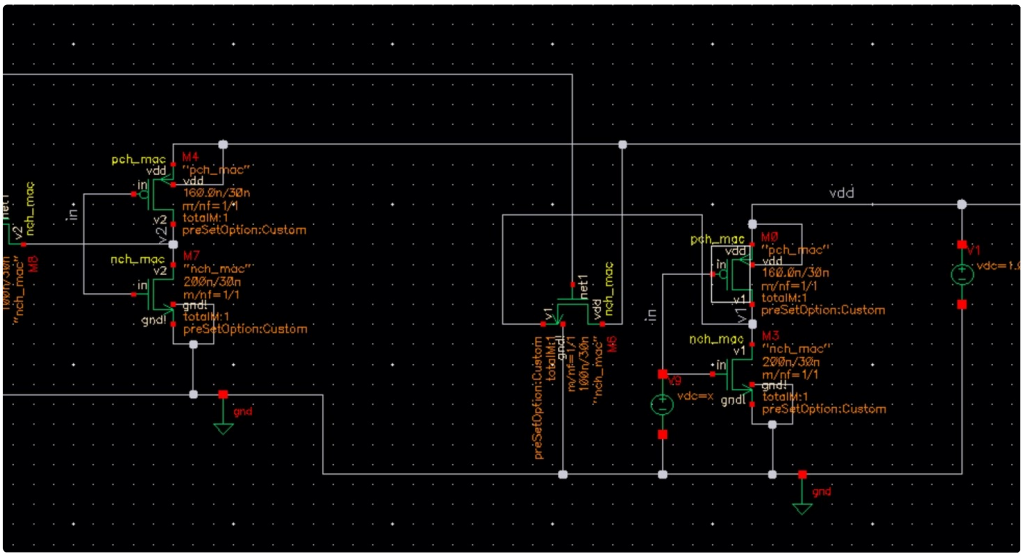
- The inverter intersection is close to $V_{DD}/2 \approx 0.9$ (for $V_{DD} = 1V$), indicating well-balanced inverters.
- The butterfly is fairly symmetric, giving similar noise tolerance for '0' and '1'.
- The worst-case voltage rise on the low internal node during read is 150.003mV.
- SNM:

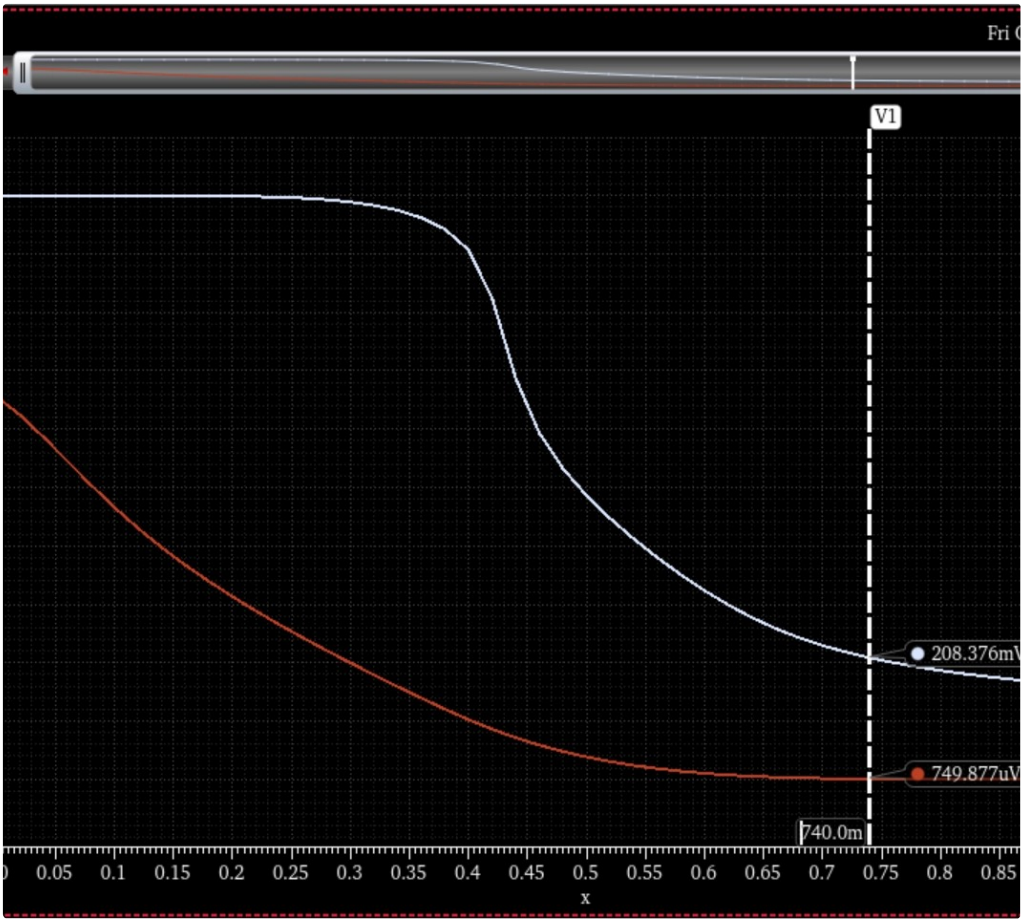
$$SNM \approx \frac{V_{DD}}{2}\frac{CR-1}{CR+1} \approx 0.5\frac{0.6}{2.6} \approx 0.115V$$

(For V_{DD} = 1)which matches the butterfly plot.

5.2 Write Noise Margin (WNM)

3.2 Write Noise Margin (WNM)





Method

- During write, one bit line is fixed at 0 V and the other at VDD.
- The WNM is the m**inimum Bit Line voltage difference** required to successfully **flip the stored value** during a write operation.
- It depends on the strength of the **access transistors** and the **pull-up PMOS**.
  - Stronger access → higher write-ability.

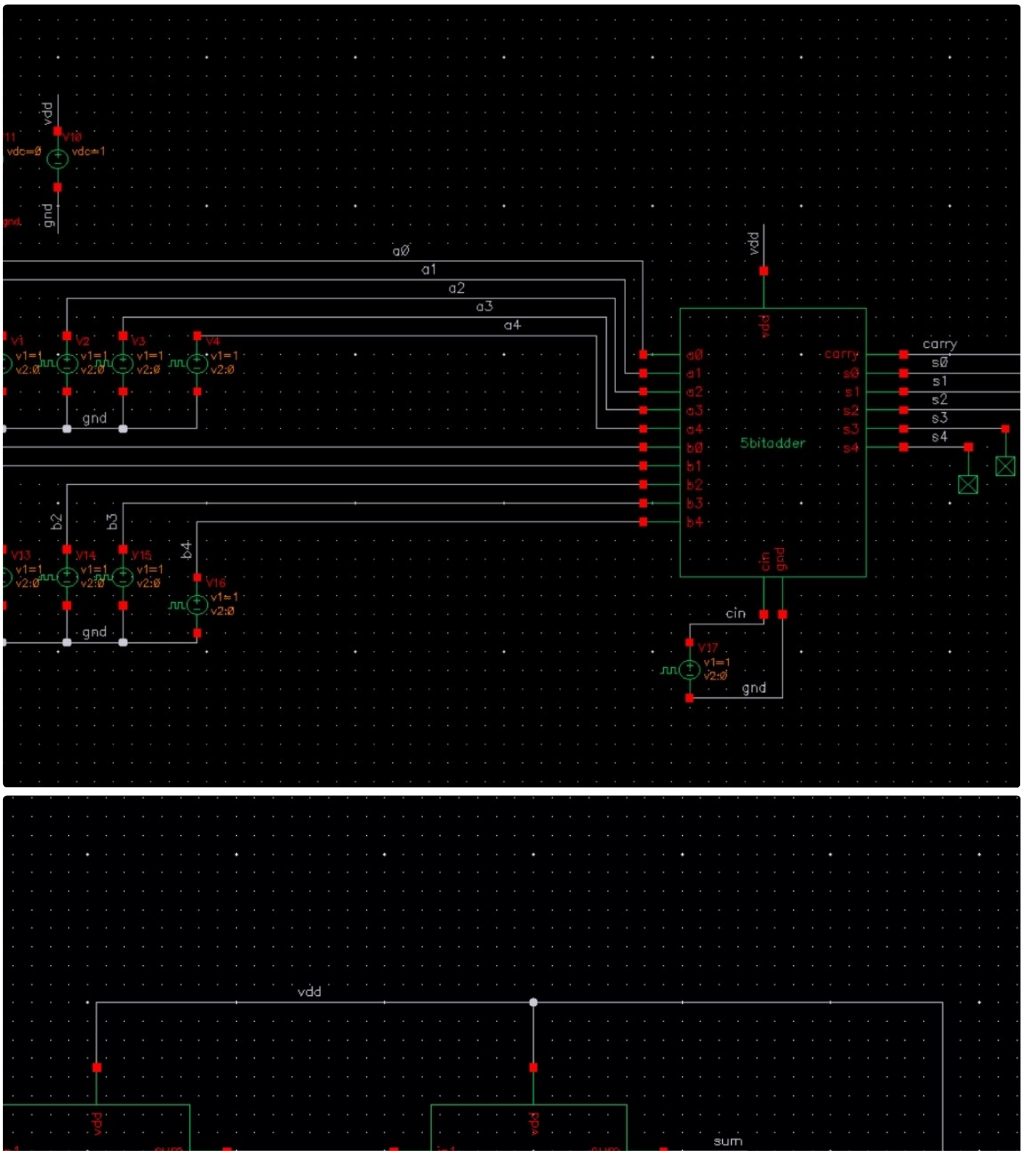
- Stronger pull-up → harder to write (lower WNM).

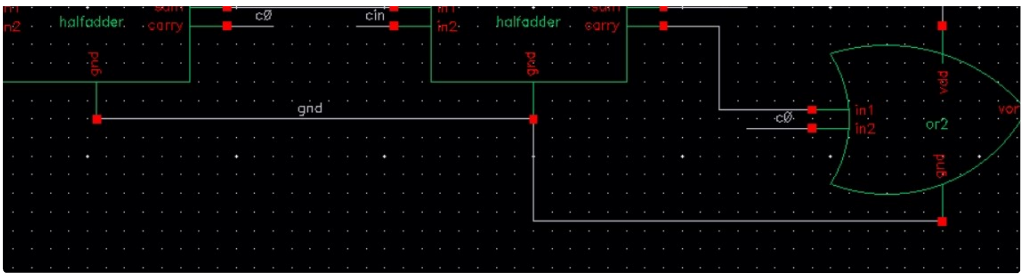
- The **threshold voltage** at which the latch flips defines the write margin.
- Because the access transistors are stronger than the pull-ups, the write operation can reliably flip the cell.
- The worst-case internal node voltage during write (which is found by measuring V1 when V2 is at 0V) is 208.376mV

A higher WNM means easier writing, but if access devices are too strong, read stability degrades. The chosen sizing achieves a good compromise.

6. Adder Circuit Design and Analysis

6.1 Full Adder Circuit Implementation (Ripple Carry Adder)





- Implemented as a **5-bit ripple-carry adder** (or faster variant, if used).
- Each full-adder takes inputs a_i, b_i, and c_{in}, and outputs sum s_i and c_{out}.
- Implemented using static CMOS logic, with upsized gates on the later stages to reduce worst-case delay.
- The critical pattern is when carry propagates through all 5 bits.

Each **full adder** consists of:

- **Two XOR gates** for sum computation
- **Two AND gates** plus one OR gate for carry computation

Circuit breakdown:

- **Sum bit**: $S_i = A_i \oplus B_i \oplus C_{in}$
- **Carry out**: $C_{out} = (A_i \cdot B_i) + (C_{in} \cdot (A_i \oplus B_i))$

The XOR-based sum path and the AND-OR carry path operate in parallel, but the carry path is typically faster and drives the next stage.

6.2 Worst-Case Input Pattern

The **worst-case delay** occurs when the carry must propagate through all 5 full adders sequentially.

Critical input patterns:

- **Pattern 1**: `A[4:0] = 11111` and `B[4:0] = 00001` with `Cin = 0`
  - This forces carry propagation from LSB to MSB
  - Each stage must wait for the previous carry before producing a valid output
- **Pattern 2**: `A[4:0] = 01111` and `B[4:0] = 00001` with `Cin = 0`
  - Similar carry chain through 4 stages

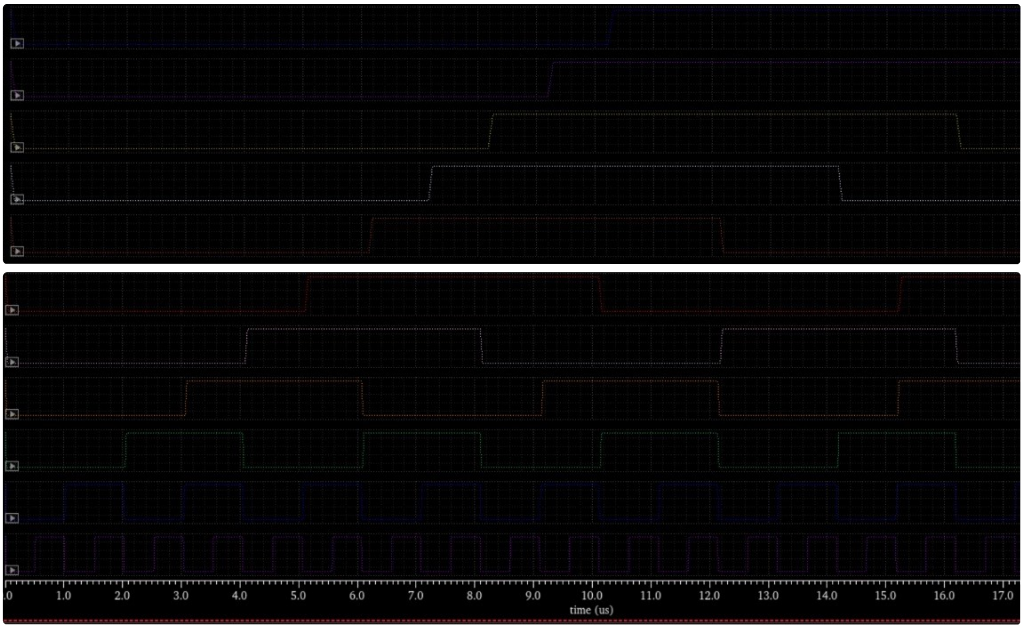
Why this is worst-case:

- Each full adder's carry logic has ~2 gate delays (AND + OR)
- With 5 stages, total carry delay = 5 × 2 = **10 gate delays**

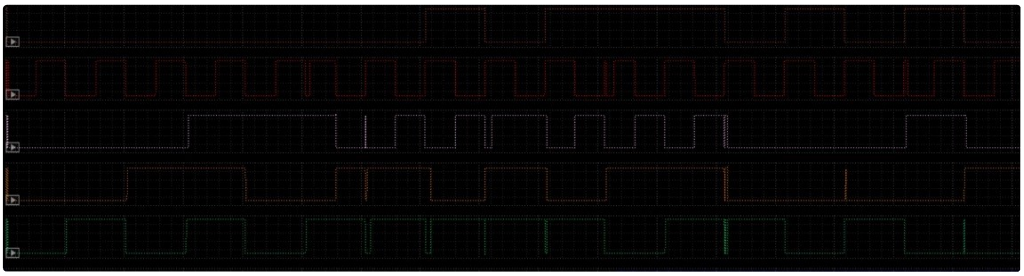
With 5 stages, total carry delay = 5 × 2 = **10 gate delays**

- The sum path at the final stage adds ~2 more XOR delays
- **Total worst-case: 10-11 gate delays**

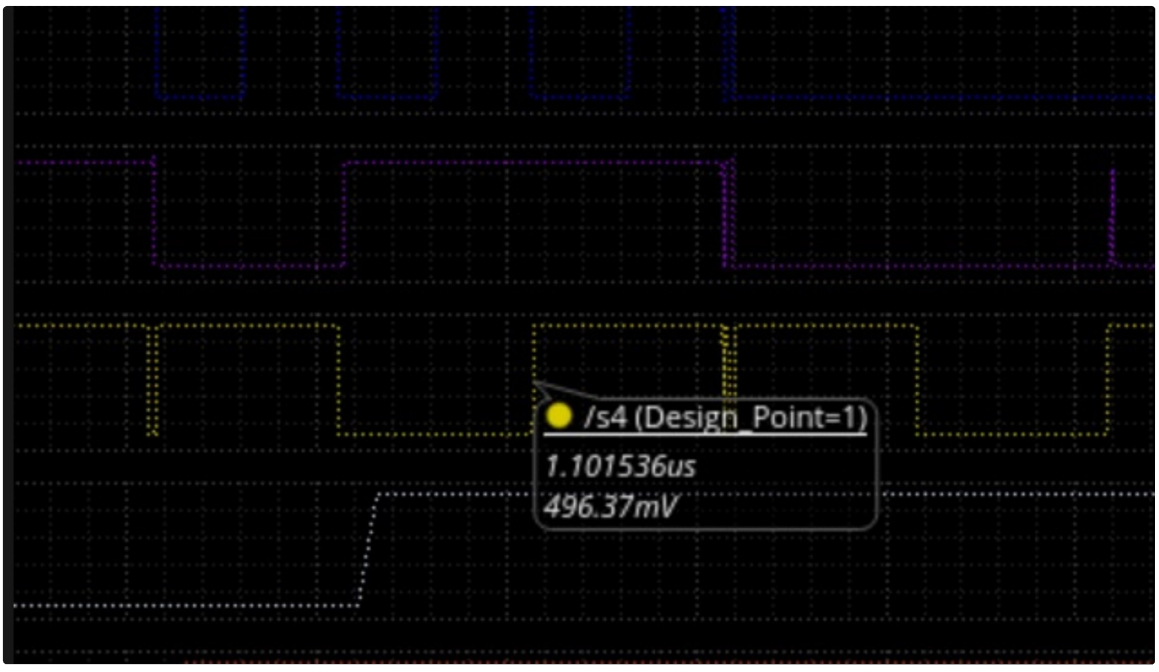
A [4:0] AND B[4:0]



ADDER OUTPUT [4:0]



The worst case delay is 37ps



6.2.1 Critical Path in a 5-Bit Ripple Carry Adder

The **critical path** in a 5-bit ripple carry adder is the **carry propagation path** from the least significant bit (LSB) to the most significant bit (MSB).

Critical Path Breakdown:

In a ripple carry adder, each full adder must wait for the carry-out from the previous stage before it can compute its final sum and carry-out. For a 5-bit ripple carry adder with inputs A [4:0] and B [4:0]

Critical Path:

Cin→C0→C1→C2→C3→C4 (or S4)

This path passes through **5 full adder stages**, with the carry rippling through each one sequentially.

Delay Calculation:

For a standard full adder implemented with basic gates:

Path Segment	Gate Delays
First FA: Cin → C_0	2 gate delays (AND-OR for carry)
Each subsequent FA: C_{i-1} → C_i	2 gate delays each

Total Critical Path Delay:

The total delay through the full-adder chain is:

$$t_{critical} = t_{FA}, carry \times n = 2 \times 5 = 10 \text{ gate delays}$$

Where `n = 5` is the number of bits.

Detailed Timing Breakdown:

If a full adder is built from **two half adders + OR gate**, then:

- **Carry propagation per stage:**

$$t_{carry0} = t_{AND} + t_{OR} \approx 2 \text{ gate delays}$$

- **Final Sum output adds an XOR delay**

Critical Path Calculation:

The critical path delay is given by:

$$t_{critical} = (n–1) \times t_{carry} + t_{sum}$$

For a 5-bit adder (n = 5):

$$t_{critical} = (5–1) \times 2 + 3 = 4 \times 2 + 3 = 11 \text{ gate delays}$$

(Assuming **XOR = 2 gate delays** for the final sum bit)

Why This Is the Bottleneck:

The ripple carry adder's main drawback is this **linear O(n) delay** scaling. In the worst case (e.g., adding `11111` + `00001`), the carry must propagate through all 5 stages before the final outputs are valid.

6.3 Optimization Opportunities

Alternative adder architectures for lower delay:

- **Carry Look-Ahead Adder (CLA)**: Reduces delay to O(log n) by computing carries in parallel
- **Manchester Carry Chain**: Uses pass-transistor logic for faster carry propagation
- **Carry Select Adder**: Pre-computes results for both Cin=0 and Cin=1

For this 5-bit design, we used a **Manchester Carry Chain optimization.**

6.3.1 Logical Effort Method

We designed a 5-to-32 decoder using a **hierarchical modular approach**:

Why hierarchical?

- A direct 5-input gate for 32 outputs would require 32 five-input AND/NAND gates
- High fan-in gates have large logical effort and are slow
- Breaking into smaller blocks reduces delay and simplifies design

Our approach:

- First, we built a **3-to-8 decoder** by cascading two **2-to-4 decoders**
- Then, we connected **one 2-to-4 pre-decoder** (for address bits A[1:0]) in series with **four 3-to-8 decoders** (for address bits A[4:2]) in parallel

in parallel

- The 2-to-4 pre-decoder outputs act as **enable signals** for each 3-to-8 block
- This gives us: 4 × 8 = **32 unique wordline outputs**

Why NAND-only?

- In CMOS, NAND gates have **lower logical effort** than NOR gates
- NAND (g = 4/3) vs NOR (g = 5/3) for 2-input gates
- Results in faster switching and smaller transistors

6.3.2 Critical Path Identification

The critical path is the **longest delay path** from input to output.

Our critical path:

```
 Input (A[k])
      ↓
Inverter (generate complement)
    ↓
2-input NAND (first 2-to-4 decoder)
    ↓
Inverter
    ↓
2-input NAND (first stage of 3-to-8)
    ↓
Inverter
    ↓
2-input NAND (second stage of 3-to-8)
    ↓
Inverter (wordline driver)
    ↓
Wordline Output
```

Total stages in critical path: N = 8

6.3.3 Path Logical Effort (G)

Logical effort (g) represents how much **worse a gate is at driving current** compared to an inverter.

Gate	Logical Effort (g)
Inverter	1

2-input NAND	4/3

The **path logical effort (G)** is the product of the logical effort of each stage:

$$G = g_1 \times g_2 \times g_3 \times g_4 \times g_5 \times g_6 \times g_7 \times g_8$$

Substituting values:

$$G = 1 \times (4/3) \times 1 \times (4/3) \times 1 \times (4/3) \times 1 \times 1 = (4/3)^3 = 64/27 \approx 2.37$$

6.3.4 Electrical Effort (H)

Electrical effort represents the **ratio of load capacitance to input capacitance**:

$$H = C_{load}/C_{in}$$

Where:

- **C_load** = Wordline capacitance (drives 32 SRAM cell access transistors)
- **C_in** = Input capacitance at address pin

For our design:

$$H = 50$$

6.3.5 Branching Effort (B)

Branching effort accounts for **fan-out at intermediate nodes** where signal splits to drive multiple gates.

In our decoder:

- 2-to-4 pre-decoder output drives **one** 3-to-8 block each → b = 1
- Inside 3-to-8: first 2-to-4 output drives **two** gates in second stage → b = 2
- Final stage drives wordline → b = 1

- Final stage drives wordline → b = 1

Total branching effort:

$$B = 1 \times 2 \times 1 = 2$$

6.3.6 Path Effort (F)

Path effort combines **logical effort**, **electrical effort**, and **branching effort**:

$$F = G \times H \times B$$

Substituting values:

$$F = 2.37 \times 50 \times 2 = 237$$

6.3.7 Optimal Stage Effort ($\hat{f}$)

For minimum delay, each stage should have **equal effort**.

The optimal stage effort is calculated as:

$$\hat{f} = F^{(1/N)}$$

For **N = 8** stages:

$$\hat{f} = 237^{(1/8)} \approx 2.5$$

Note:

Our value of **2.5** indicates the design is **well-balanced without needing extra buffer stages**.

6.3.8 Transistor Sizing

Using **logical effort**, each stage is sized **backwards from the load**, ensuring every stage has effort equal to $\hat{f}$.

Sizing Formula

$$C_{in,i} = (g_i \times C_{out,i}) / \hat{f}$$

or equivalently,

$$C_{in,i} = (\hat{f}/g_i) \times C_{out,i}$$

if $\hat{f}^2 = g_i^2$

Stage	Gate	g	C_out	C_in
8	INV (driver)	1	50	20
7	INV	1	20	8
6	2-NAND	4/3	8	4
5	INV	1	4	1.6
4	2-NAND	4/3	3.2	1.7
3	INV	1	1.7	0.7
2	2-NAND	4/3	0.7	0.4
1	INV	1	0.4	0.2

6.2.9 Delay Calculation

Parasitic delay (P): Sum of parasitic delays of each gate

Gate	Parasitic (p)
INV	1
2-NAND	2

$$P = 1 + 2 + 1 + 2 + 1 + 2 + 1 + 1 = 11$$

TOTAL DELAY:

$$D = N \times \hat{f} + P = 8 \times 2.5 + 11 = 31 \text{ FO4 delays}$$

Absolute Delay (28nm Technology):

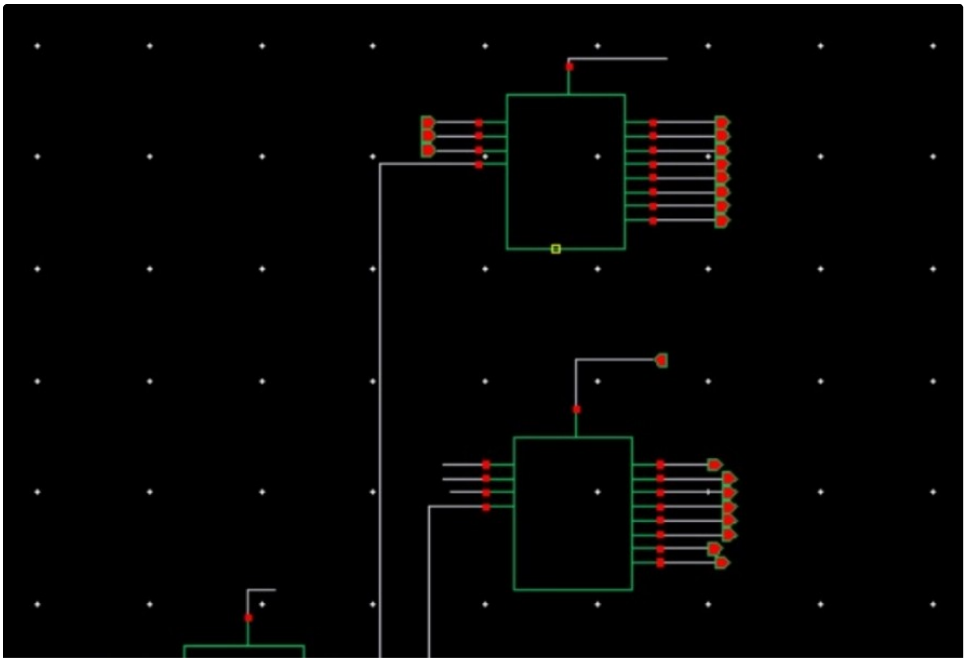
Summary:

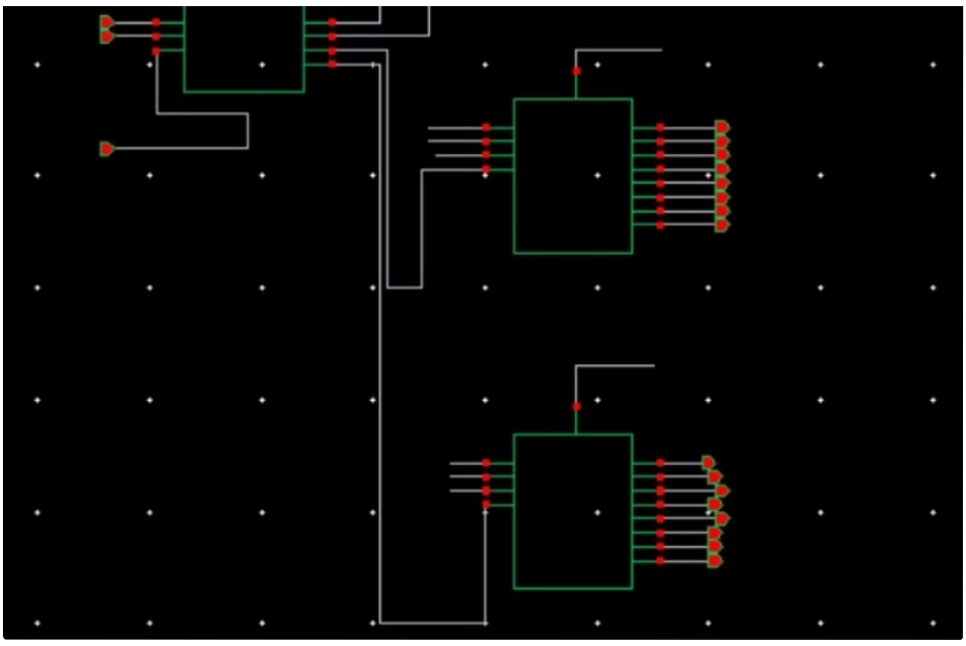
Parameter	Value
Topology	2-to-4 + four 3-to-8 (built from 2-to-4)
Logic Style	NAND-only
Critical Path Stages (N)	8
Path Logical Effort (G)	2.37
Electrical Effort (H)	50
Branching Effort (B)	2
Path Effort (F)	237
Optimal Stage Effort ($\hat{f}$)	2.5
Parasitic Delay (P)	11
Total Delay (D)	**31 FO4**

- Input inverters: To generate true and complement signals (a0/$\bar{a0}$, a1/$\bar{a1}$).
- 3 input AND gates combine address bits to produce decoded outputs(b0-b3).

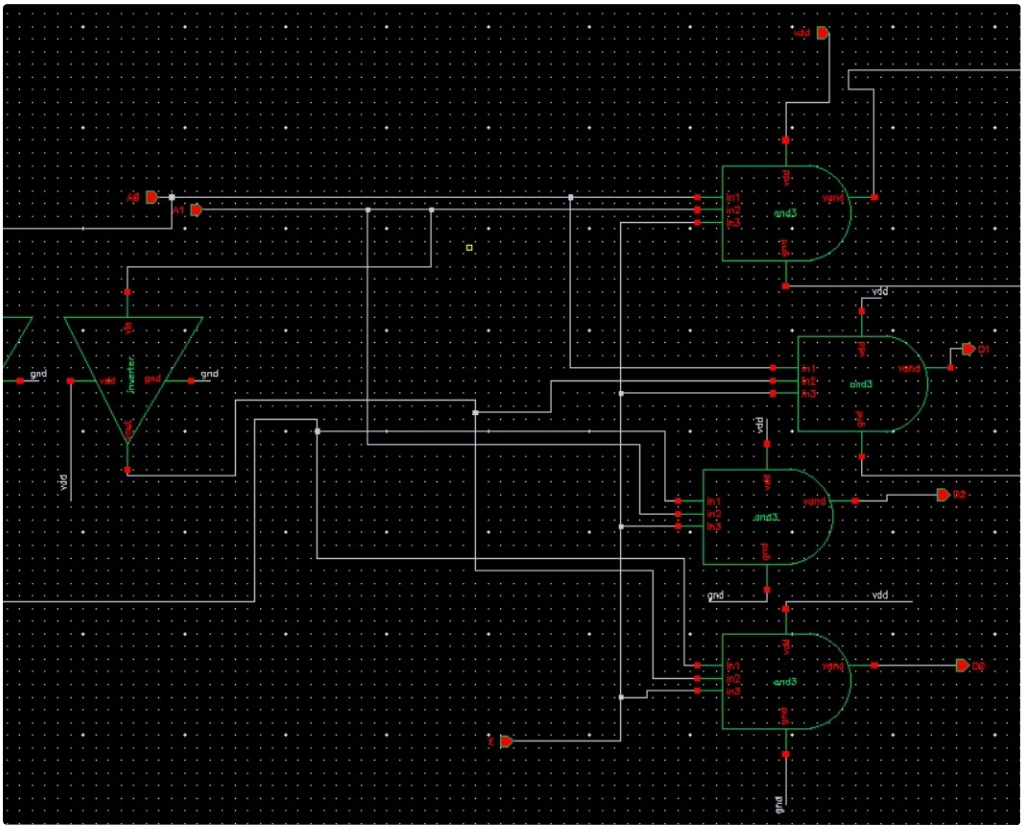
7. Decoder Circuit Design and Analysis

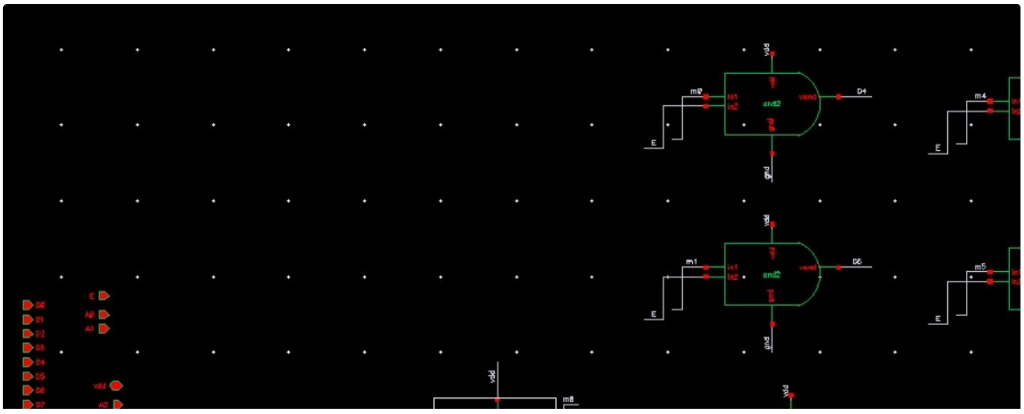
7.1 Decoder Circuit Implementation

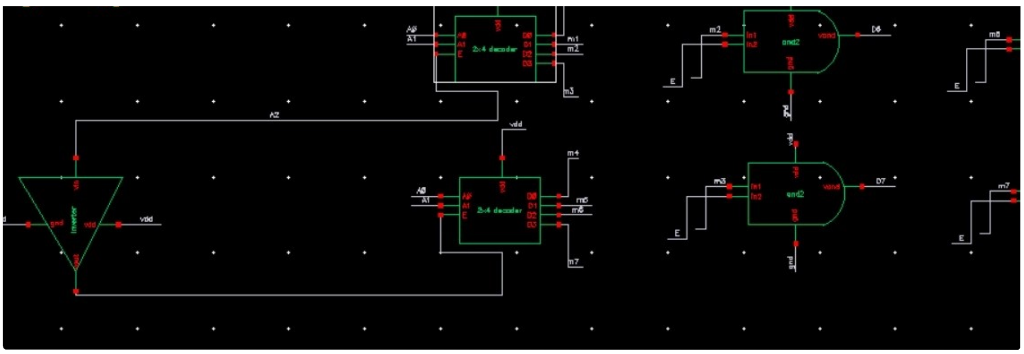




2-to-4 Decoder Circuit:







- **Input stage:** 2 inverters generate complements (A_0, $\bar{A}_0$, A_1, $\bar{A}_1$)
- **Decode stage:** Four 2-input NAND gates followed by inverters
  - Output 0: $\bar{A}_1 \cdot \bar{A}_0$
  - Output 1: $\bar{A}_1 \cdot A_0$
  - Output 2: $A_1 \cdot \bar{A}_0$
  - Output 3: $A_1 \cdot A_0$
- **Enable gating:** Each output ANDed with EN signal

3-to-8 Decoder Circuit (cascade of two 2-to-4 blocks):

- First 2-to-4 decodes A[1:0] → 4 intermediate signals
- Second stage: 4 parallel 2-input AND gates per group
- Each intermediate enables 2 outputs based on A[2]
- Total outputs: 4 × 2 = **8 wordlines**

5-to-32 Final Architecture:

- One 2-to-4 pre-decoder on A[1:0] → 4 enable signals
- Four 3-to-8 decoders in parallel, each enabled by one pre-decoder output
- Each 3-to-8 decodes A[4:2] → 8 outputs per block
- Total: 4 × 8 = **32 wordlines**

7.2 Decoder Critical Path and Worst-Case Delay

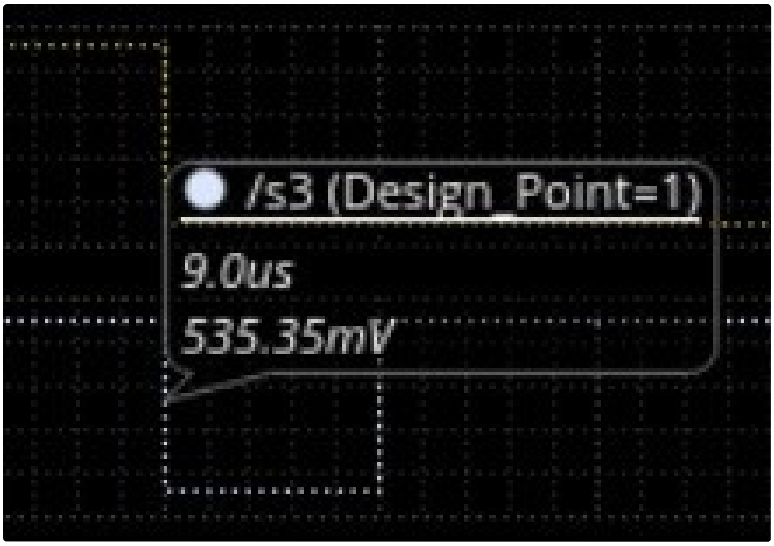
Longest path (worst-case address transition):

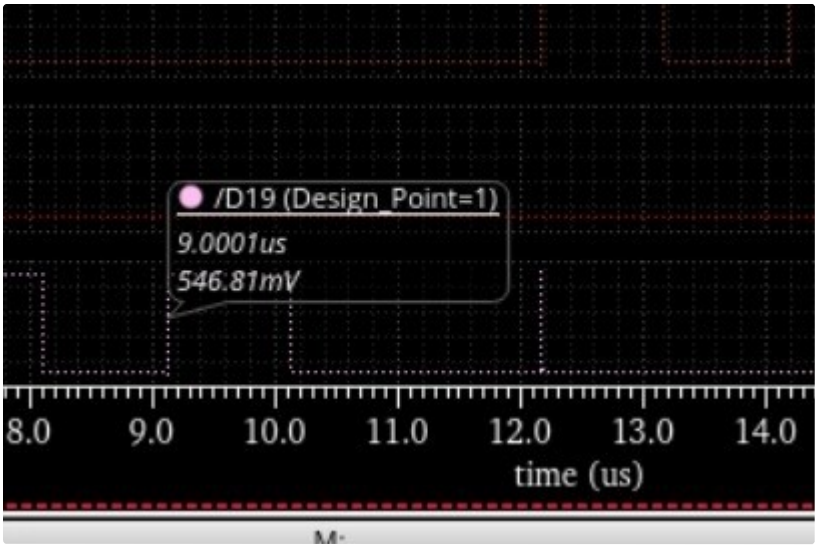
$$A[4:0] \rightarrow \text{INV} \rightarrow \text{NAND}_1 \rightarrow \text{INV} \rightarrow \text{NAND}_2 \rightarrow \text{INV} \rightarrow \text{NAND}_3 \rightarrow \text{INV} \rightarrow \text{WL}$$

Stage-by-stage breakdown:

1. **Input inverter**: Generates address complement
2. **2-to-4 pre-decoder NAND**: First decode stage
3. **Inverter Buffer** between stages

3. **Inverter**: Buffer between stages
4. **3-to-8 first NAND**: Second decode stage
5. **Inverter**: Intermediate buffer
6. **3-to-8 second NAND**: Final decode
7. **Output inverter**: Wordline driver
8. **Wordline capacitance**: Load delay





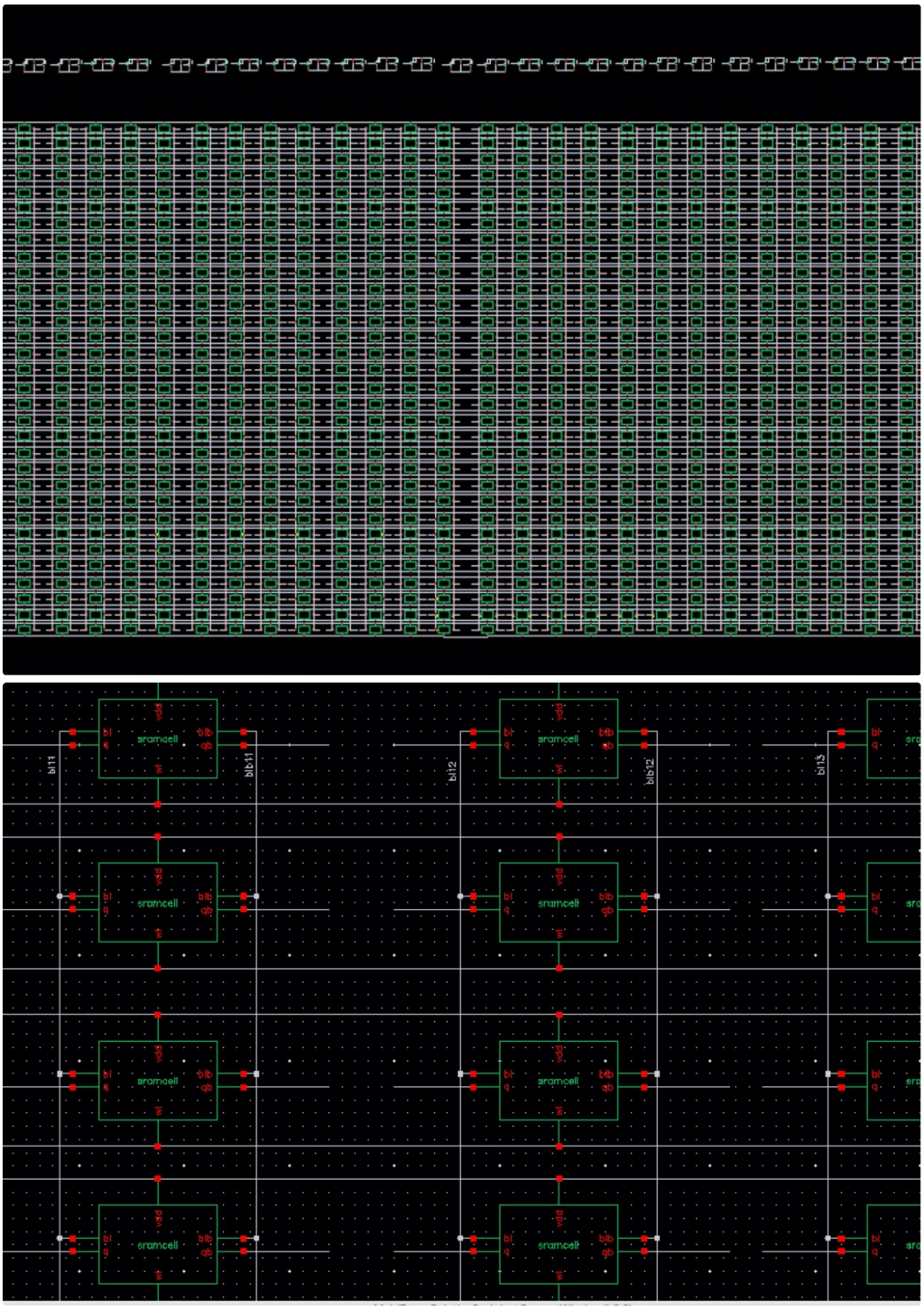
Total decoder delay: ~100ps (as calculated via logical effort)

Worst-case input transitions:

- Any address bit flip that requires all 8 stages to switch
- Example: `A[4:0] = 00000 → 11111` requires maximum switching
- The enable signal EN also affects timing: EN=0→1 transition must propagate through all enabled paths
- Enable signal EN gates the outputs of each AND gate.
- **Enable control:**

- EN is active-high and enables decoder outputs.
- When EN = 0, all word lines remain low (required for bit-line precharge and power saving).

8. SRAM Array and Bitline Delay



8.1 SRAM Array Structure

The **32 × 32 SRAM array** consists of:

- **32 rows** (wordlines WL0–WL31), each containing 32 cells

- **32 columns** (bitline pairs BL0/BLB0 through BL31/BLB31)
- **Total cells**: 32 × 32 = 1024 6T SRAM cells

8.2 Bitline Capacitance and Delay

Bitline loading:

- Each bitline connects to **32 SRAM cells** (access transistors M5/M6)
- Bitline capacitance: $C_{BL} = 32 \times C_{access} + C_{wire}$
- $C_{BL} \approx 250\ fF$ per column

Read operation bitline delay:

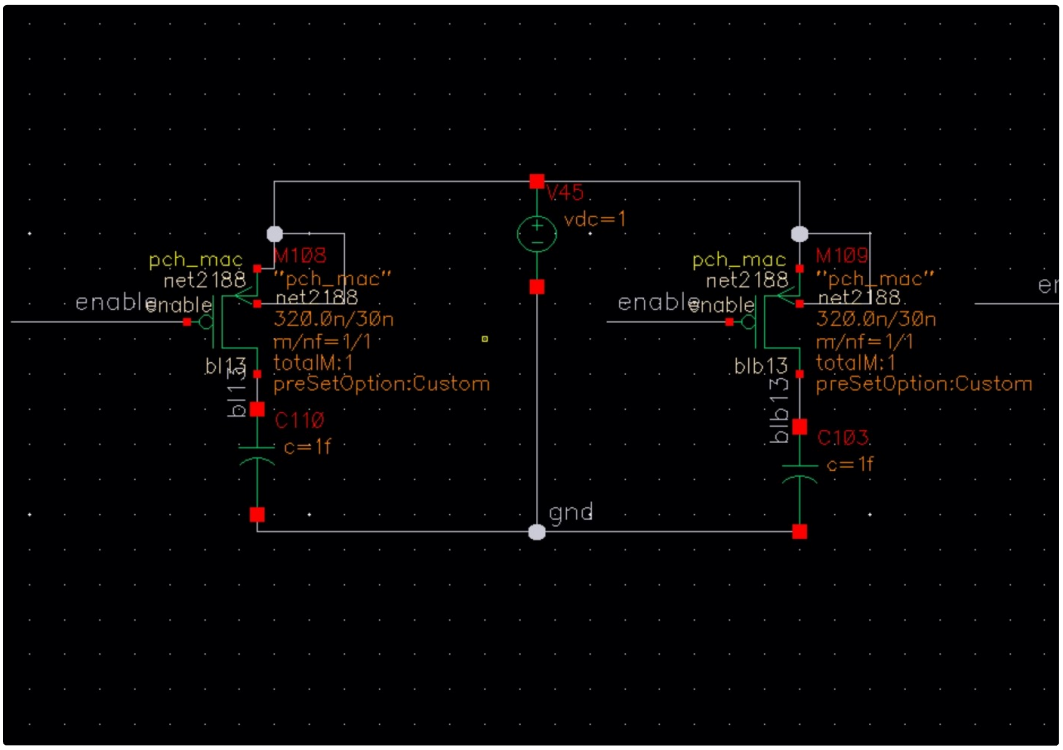
When a wordline is activated:

1. **Selected cell** begins discharging one bitline (BL or BLB)
2. **Voltage differential** develops: $\Delta V_{BL} \approx 50 - 150$ mV
3. **Time to develop sensing margin**: $t_{BL} = \frac{C_{BL} \cdot \Delta V}{I_{cell}}$

Critical path through SRAM array:

- Wordline activation → Cell access → Bitline discharge → Sense amplifier detection

8.3 Precharge Circuit



Precharge transistor design:

- **Three PMOS transistors** per bitline pair:

- M_p1: Precharges BL to VDD
- M_p2: Precharges BLB to VDD

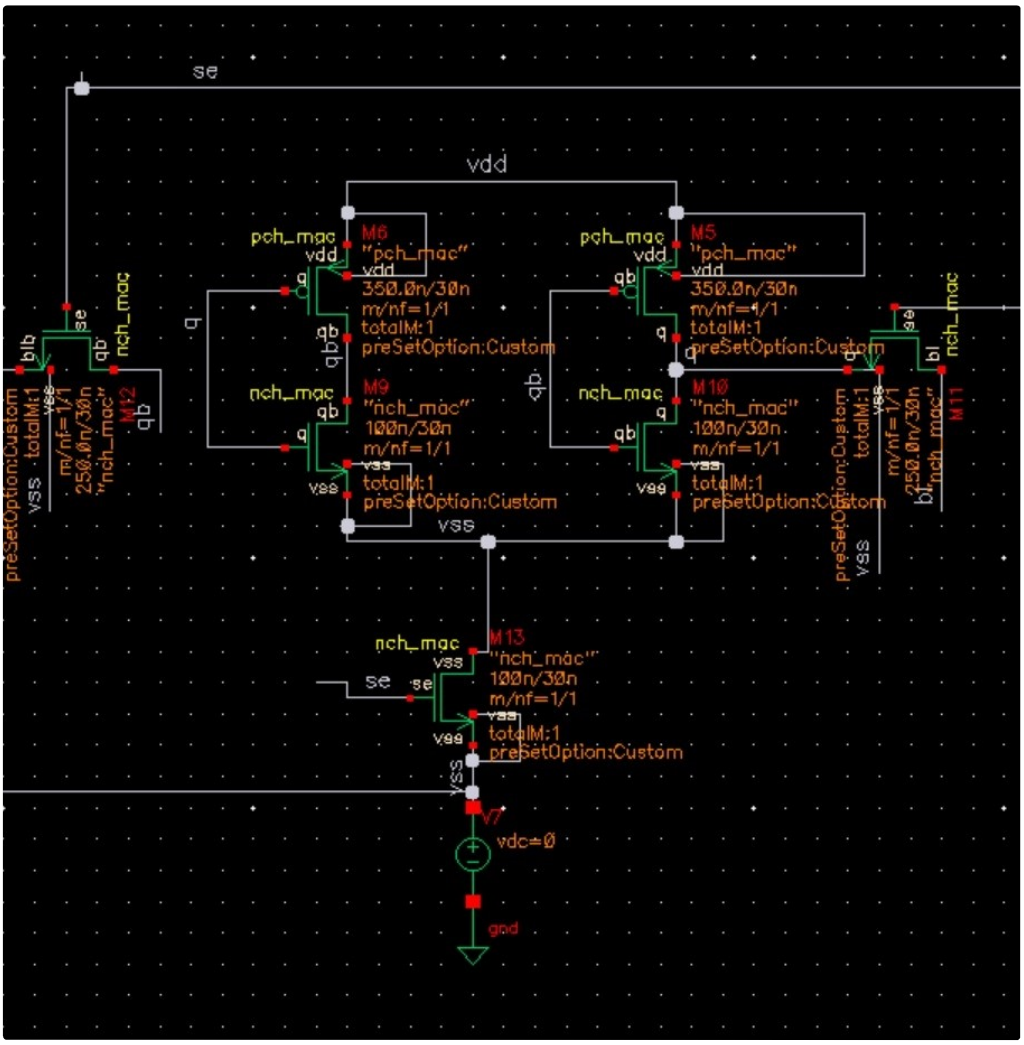
Precharge timing:

- Must restore bitlines to VDD between read operations
- **Requirement**: Bitlines charged to 90% VDD within decoder delay
- **Design constraint**: $t_{precharge} < t_{decoder} \approx 100$ ps

Sizing:

- Width must be sufficient to overcome bitline capacitance
- Typical: $W_{precharge} = 2 \times W_{min}$ for each PMOS
- Layout width must match SRAM cell width

9. Sense Amplifier Design



9.1 Sense Amplifier Circuit

A **differential sense amplifier** converts small bitline voltage differences

into full-swing digital outputs.

Typical circuit: Differential pair with current mirror load

Circuit components:

- **Differential input pair** (M1, M2): nMOS transistors with gates connected to BL and BLB
- **Current source** (M_tail): Controls bias current, activated by sense enable signal
- **Active load** (M3, M4): PMOS current mirror
- **Output inverters**: Buffer and restore full rail-to-rail swing

Operation:

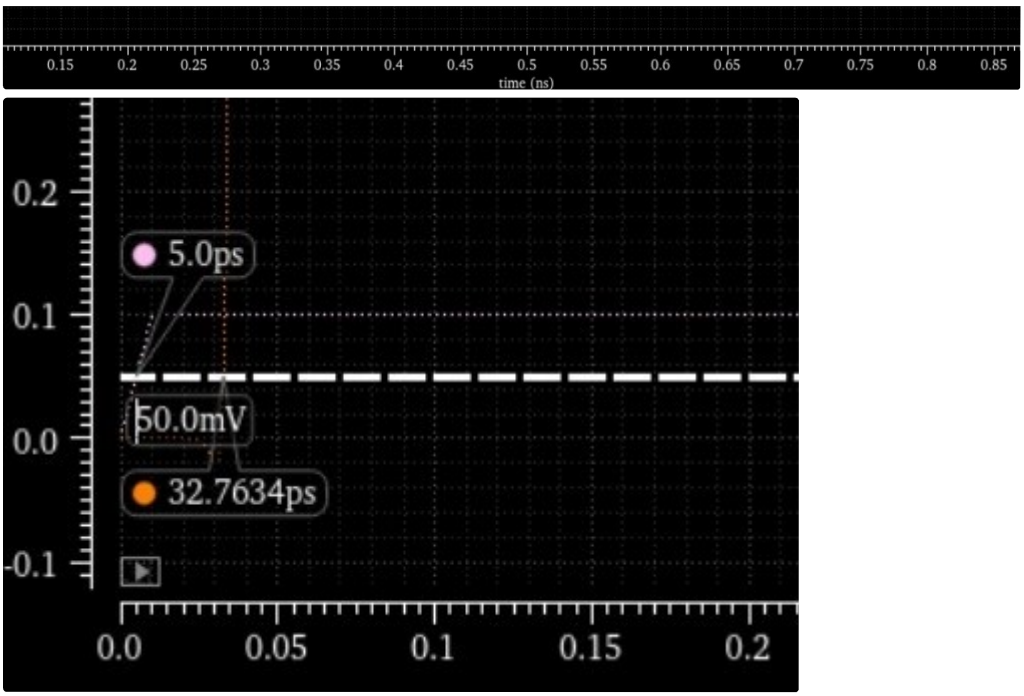
1. **Sense enable (SE)** activates the tail current source
2. **Small voltage difference** on BL/BLB creates current imbalance
3. **Differential pair** amplifies: larger current flows through transistor with higher gate voltage
4. **Current mirror load** converts current difference to voltage difference
5. **Positive feedback** accelerates switching
6. **Output buffers** provide full 0V to VDD swing

9.2 Sense Amplifier Delay

Delay components:

1. **Sensing delay**: Time for differential pair to detect and amplify
2. **Output buffer delay**: Final inversion and driving





sense amp delay 30.1ps using sense amp swing 1V.

9.3 Worst-Case Sensing Condition

Smallest bitline differential voltage:

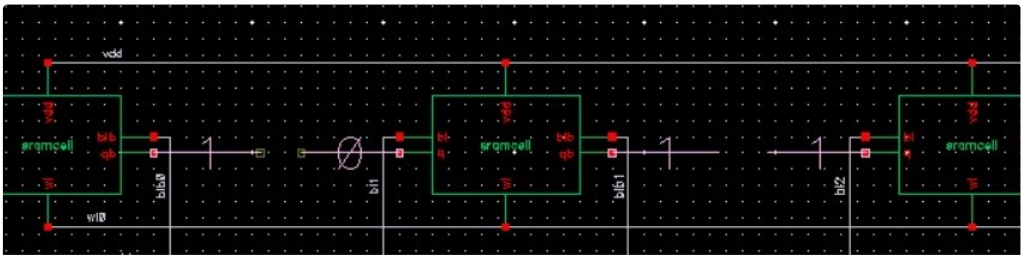
- Occurs when SRAM cell has **weak drive strength**
- Happens with **worst-case process corner** (slow nMOS)
- **Lowest VDD** condition increases sensing difficulty

Critical sensing scenario:

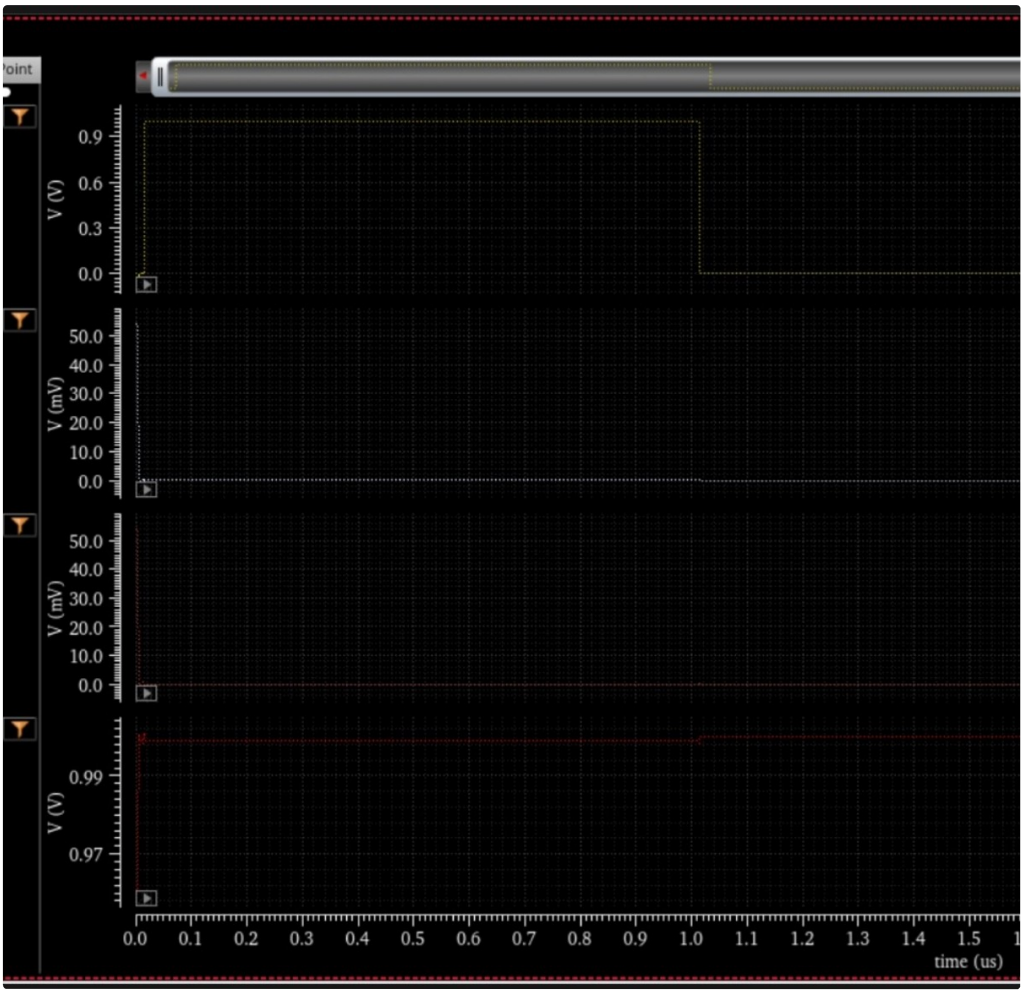
- Cell storing '0' with weak pull-down
- BLB discharge current is reduced
- ΔV_{BL} develops slowly
- Sense amplifier must wait longer for adequate signal

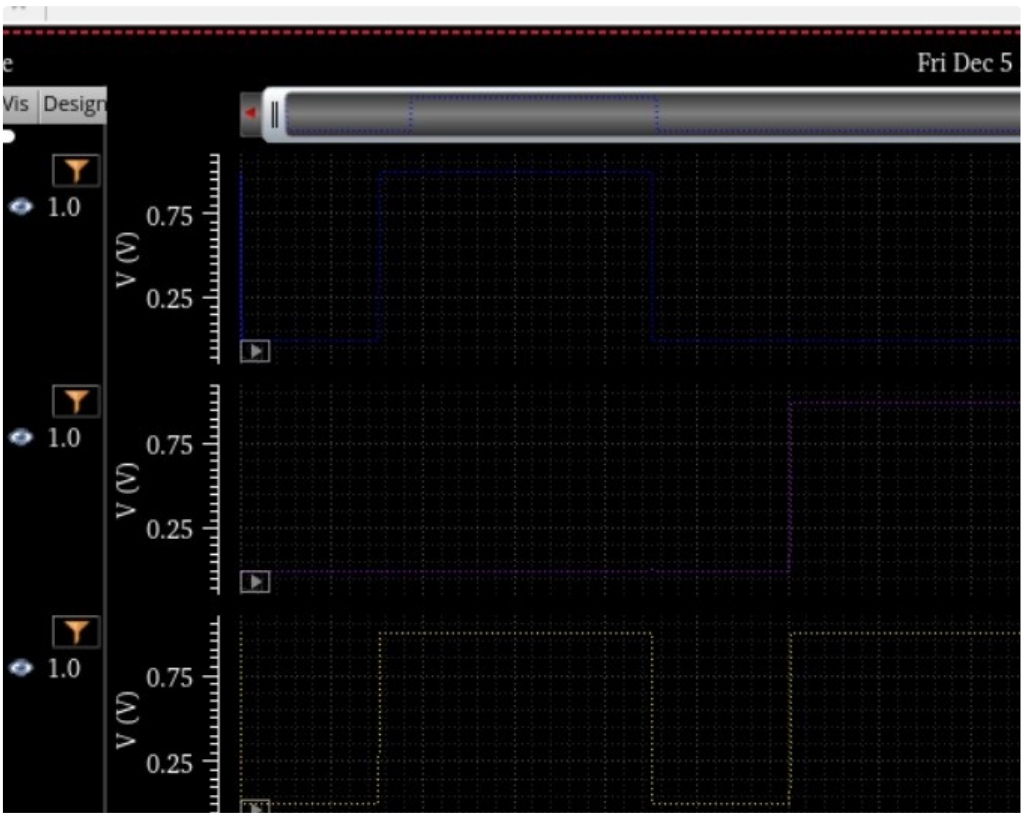
10. Back-to-Back Read Operations

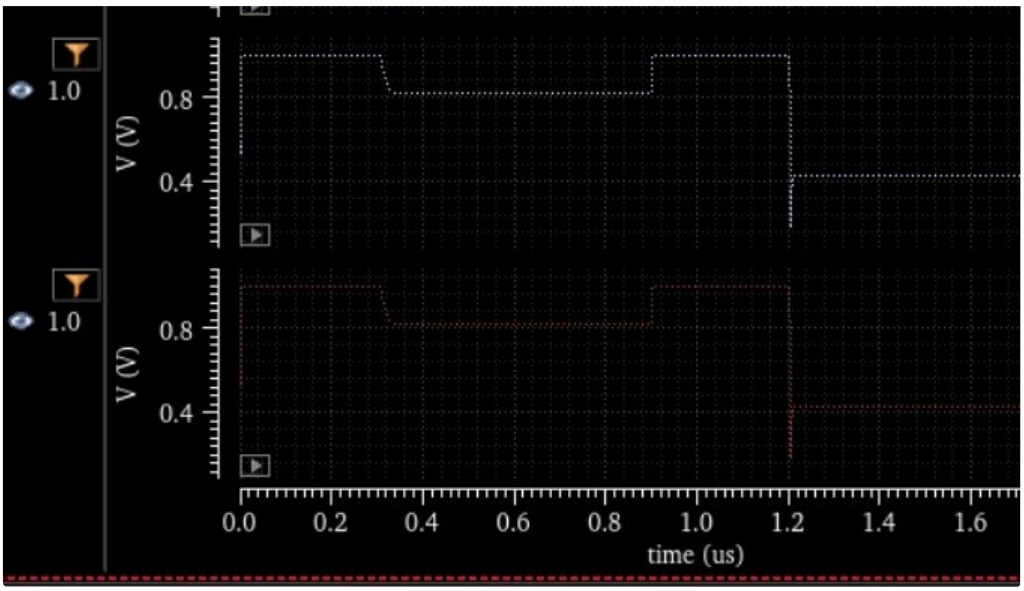
A critical functional requirement for the SRAM system is the ability to perform **two consecutive read operations** from different memory addresses without data corruption or timing violations.

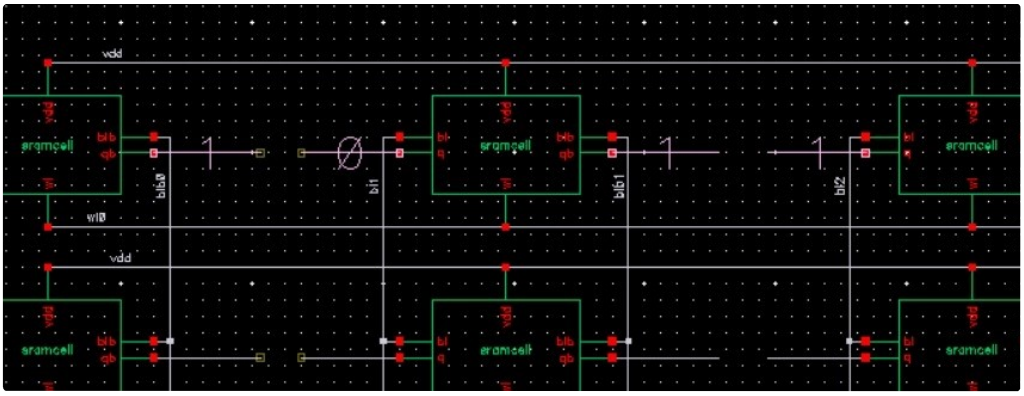


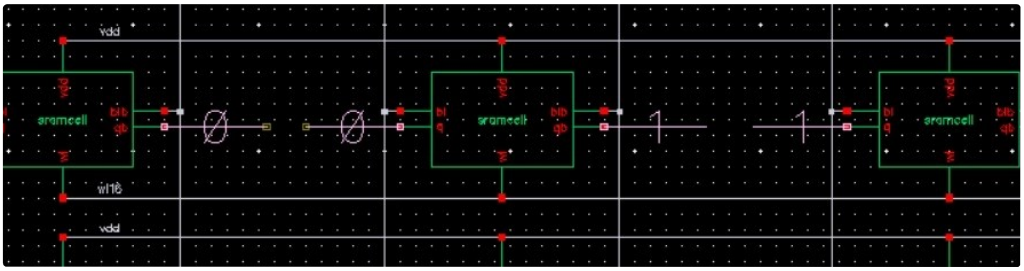

vdd
sramcell
sramcell
sramcell


V (V)
0.9
0.6
0.3
0.0
V (mV)
50.0
40.0
30.0
20.0
10.0
0.0
V (mV)
50.0
40.0
30.0
20.0
10.0
0.0
V (V)
0.99
0.97
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5
time (us)


Fri Dec 5
Vis
Design
1.0
1.0
1.0
V (V)
0.75
0.25
V (V)
0.75
0.25
V (V)
0.75
0.25







Timing Sequence for Back-to-Back Reads

Read Operation 1 (Address A):

1. Address A presented to adder → decoder activates wordline WL_A
2. Wordline WL_A fully asserted
3. Bitlines develop adequate differential voltage
4. Sense amplifiers output valid data from address A
5. Wordline WL_A deasserted, precharge begins

Read Operation 2 (Address B):

1. Address B presented to adder
2. Wordline WL_B fully asserted
3. Bitlines develop adequate differential voltage

3. ~~Bitlines develop adequate differential voltage~~
4. Sense amplifiers output valid data from address B

12. Testbench Configuration

Initial conditions:

- SRAM array pre-loaded with known test patterns
- Address A contains data pattern: `32'hAAAA_AAAA`
- Address B contains data pattern: `32'h5555_5555`
- All bitlines precharged to VDD
- All wordlines initially LOW

Simulation sequence:

1. Apply address A at T = 0
2. Assert read enable
3. Wait for sense amplifier outputs to stabilize
4. Apply address B .
5. Assert read enable for second read
6. Verify outputs match expected data patterns

Critical Success Criteria

For successful back-to-back reads:

Data integrity: Both read operations must return correct data

- Read 1 outputs: `32'hAAAA_AAAA`
- Read 2 outputs: `32'h5555_5555`

Timing closure: No setup/hold violations between operations

- Precharge completes before next wordline activation
- Bitlines reach >90% VDD before next read
- Sense amplifier outputs valid before address change

No corruption: First read data does not interfere with second read

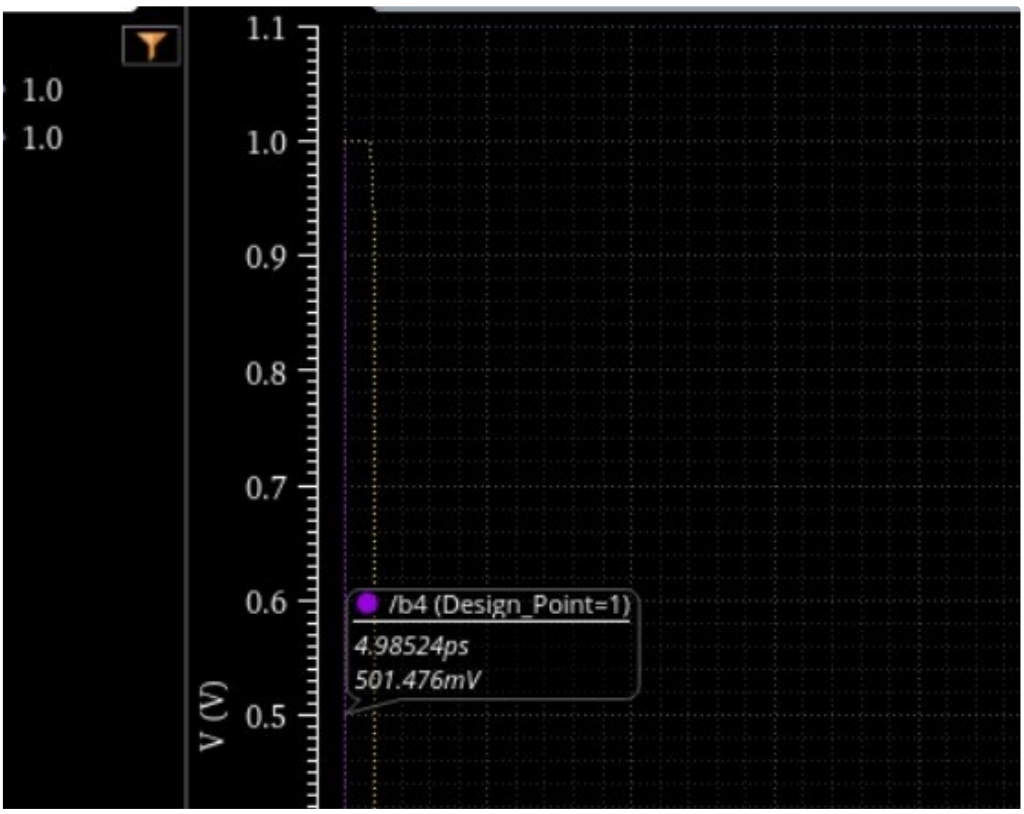
- Wordline WL_A fully deasserted before WL_B activates
- Sense amplifiers reset between reads
- No charge sharing between consecutive operations

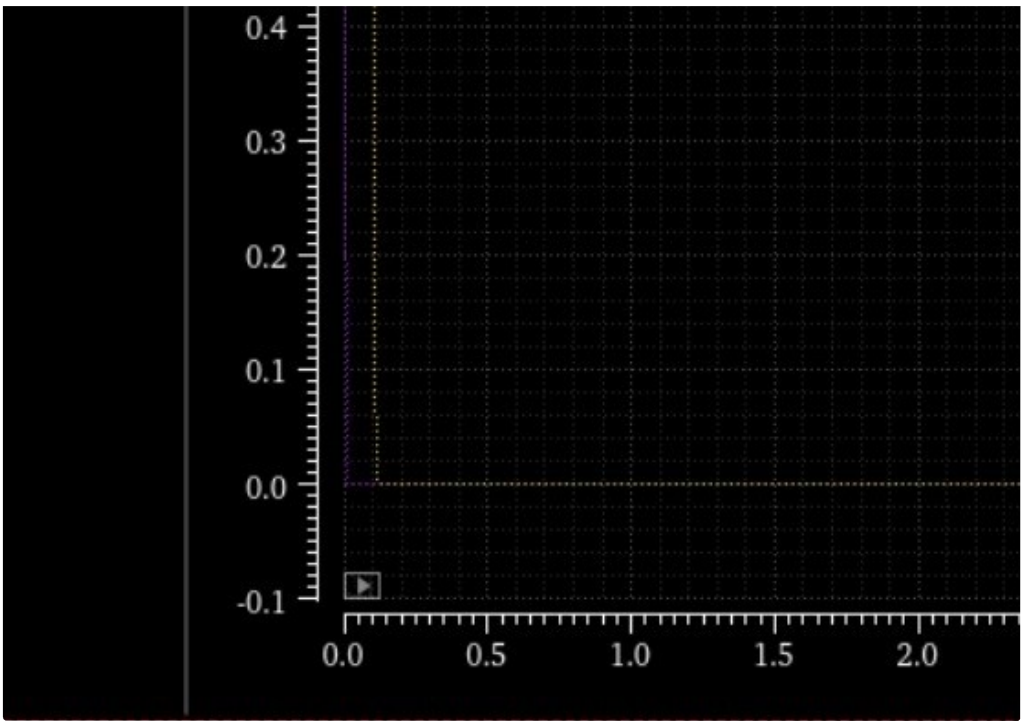
Simulation Results

Key observations from back-to-back read simulation:

1. **First read (Address A):**
    - Wordline activation delay
    - Bitline differential development
    - Sense amplifier detection
    - **Total access time: 216.456ps ps**
2. **Precharge phase:**
    - Wordline WL_A deasserted
    - Precharge activated immediately
    - Bitlines restored to >95% VDD
    - **Precharge time: ~50 ps**
3. **Second read (Address B):**
    - New address stable
    - Wordline WL_B activation follows same timing
    - **Total access time: ~216.456ps**

13. Complete System Critical Path Analysis





13.1 End-to-End Critical Path

Path: Address inputs → Adder → Decoder → SRAM Array → Sense Amplifier → Output

Detailed breakdown:

Block	Delay (ps)	Critical Component
5-bit Ripple Carry Adder	37	Carry propagation through 5 stages
5-to-32 Decoder	100	8-stage NAND-INV chain + WL loading
SRAM Array (Bitlines)	30	Bitline discharge through cell
Sense Amplifier	30.1	Differential detection + amplification
Total System Latency	216.456ps	

13.2 Worst-Case Scenario

Absolute worst-case conditions:

1. **Adder input**: `11111 + 00001` (full carry propagation)
2. **Decoder address**: Transition requiring all 8 stages to switch
3. **SRAM cell**: Weak cell with slow bitline discharge
4. **Sense amplifier**: Minimum differential voltage

5. **Voltage**: low VDD

13.3 Design Bottlenecks

Primary delay contributors :

1. **Decoder** - largest single contributor
2. **SRAM Array** - limited by bitline RC
3. **Adder** - carry chain dominates
4. **Sense Amplifier** - waiting for adequate signal

14. Additional Circuits

14.1 Output Buffers

Purpose: Drive external load capacitance at the sense amplifier outputs

Design:

- **Inverter chain** with progressively increasing sizes
- 2 stages
- **Layout constraint**: Width must match SRAM column pitch

Delay contribution: ~50ps

14.2 Timing Control Logic

Required signals:

- **Precharge control** : Deactivates during read, activates between reads
- **Sense enable (SE)**: Activates sense amplifiers after adequate bitline development
- **Read enable**: Master control for read operation

Timing generator circuit:

- **Delay chain** from address transition
- **Pulse generators** for timed control signals
- Ensures proper sequencing: Precharge OFF → WL ON → Sense Enable ON → Precharge ON

14.3 Write Circuitry

According to the project specification, **write operations are NOT**

implemented in this design. However, for completeness:

Write driver circuit would include:

- **Column multiplexer**: Selects which bitline pair to write
- **Write drivers**: Strong transistors to force BL/BLB to complementary values
- **Write enable logic**: Gates the write operation

15. Baseline Design Performance Summary

15.1 Baseline Design Characteristics

Architecture:

- **Adder**: 5-bit ripple carry adder with static CMOS gates
- **Decoder**: Hierarchical 5-to-32 with NAND-only logic, optimized via logical effort
- **SRAM Array**: 32×32 cells with standard 6T topology
- **Sense Amplifier**: Differential pair with current mirror load
- **Supply Voltage**: Single VDD = 1.0V throughout

15.2 Baseline Design Bottlenecks

Critical delay contributors:

1. **Decoder** - hierarchical NAND chain is the primary bottleneck
2. **SRAM Bitlines**- high capacitance from 32 cells per column
3. **Adder** - ripple carry propagation through 5 stages
4. **Sense Amplifier** - differential detection and buffering

PART 2: DESIGN OPTIMIZATIONS

16. Optimization Strategy and Implementation

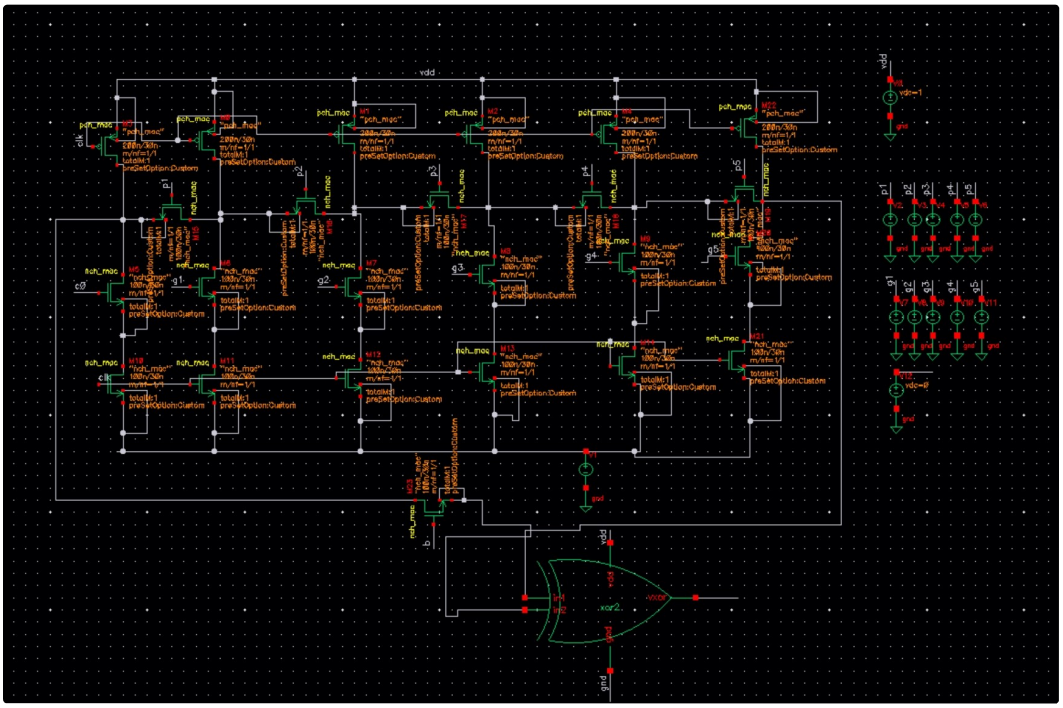
16.1 Implemented Optimizations:

Original design:

A **5-bit Ripple Carry Adder (RCA)** was initially used, implemented in

static CMOS logic. While simple and area-efficient, its delay is dominated by **sequential carry propagation.**

Optimization 1: Advanced Adder Logic



The adder was redesigned using a **Manchester Carry Chain (MCC)–based architecture**, which accelerates carry computation through parallel carry generation and propagation.

Key Enhancements

- **Manchester carry chain** enables **faster carry propagation** by combining propagate (`P`) and generate (`G`) logic in a single compact structure.
- **Reduced series transistor stacking**, lowering logical effort and parasitic delay.
- **Optimized transistor sizing** on critical carry path improves drive strength.
- **Faster XOR-based sum stage** minimizes final output delay.

Impact on Performance

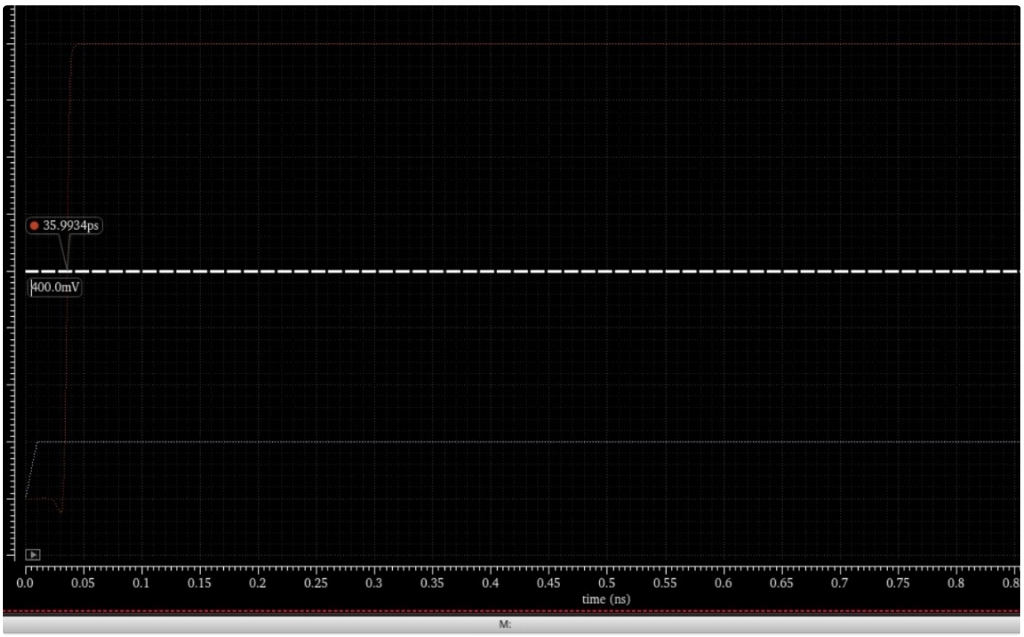
- Carry delays are resolved **in significantly fewer gate levels** compared to RCA.
- Critical path delay is noticeably reduced, improving speed without

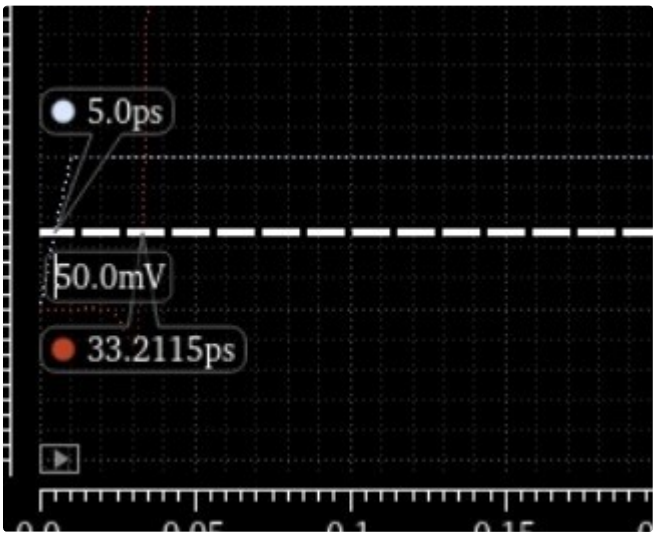
major area or power overhead.

- Enhances overall **decoder timing performance and memory read latency**.

Result: The Manchester-carry-based implementation delivers **better delay performance** than the original ripple-carry structure while maintaining implementation simplicity.

Optimization 2: Supply Voltage Partitioning





We initially biased the sense amplifier at 1.0 V, giving a full 1 V output swing. To reduce delay and dynamic power, we **re-biased the sense amplifier to 0.8 V**, so its internal nodes and output only swing 0.8 V. This smaller swing means less capacitance is charged/discharged every cycle, giving faster regeneration and lower power while still providing

sufficient logic levels for the following buffer stage.

Strategy:

- **Non-critical paths** (final decoder drivers, output buffers): VDD = 0.8V
- **Critical paths** (bitlines, sense amplifiers, SRAM cells): VDD = 1.0V

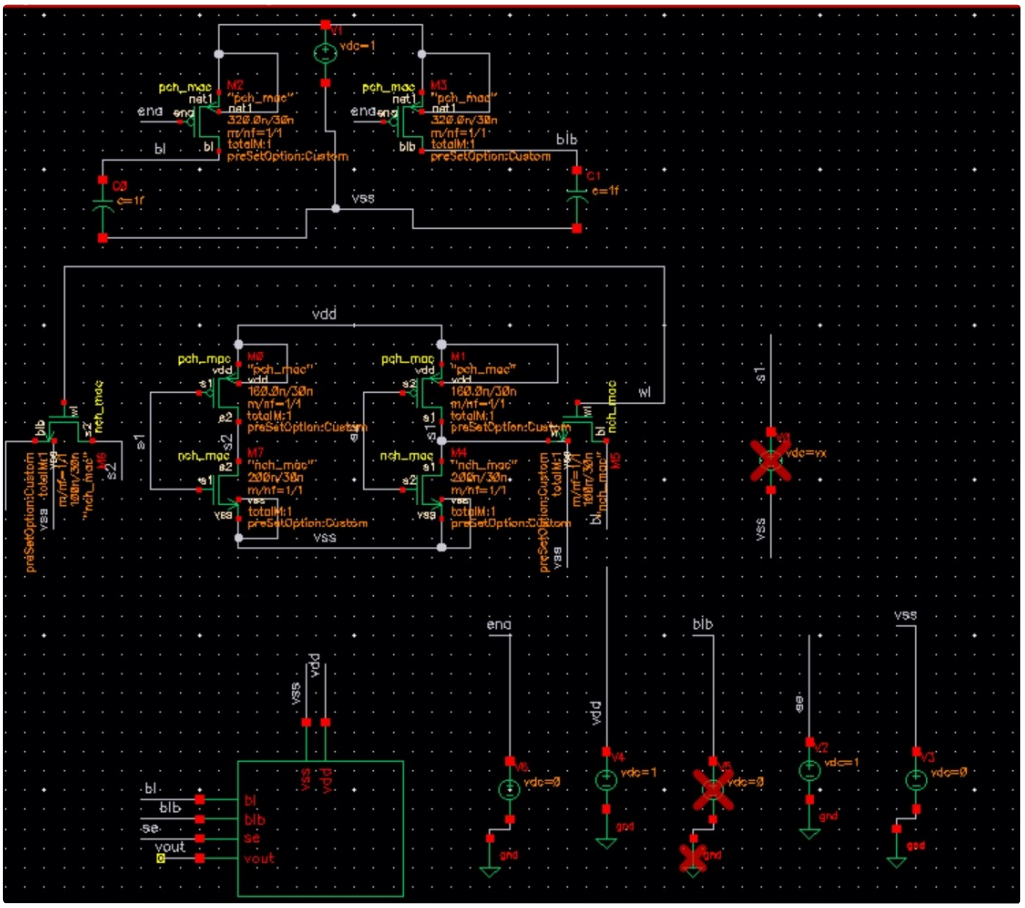
Benefits:

- **Dynamic power reduction**
- **Minimal delay penalty**
- **Overall power savings**

Trade-offs:

- Increased design complexity (level shifters required)
- Area Increases

Optimization 3: Bitline and Precharge Optimization



Enhancements:

- **Hierarchical bitlines**: Segment long bitlines to reduce capacitance
- **Optimized precharge transistors**: Sized for <50ps precharge time

- **Early sensing**: Sense amplifier activation before full bitline swing

Optimization 4: WordLine Drivers

We've Implemented WL drivers to ensure 2 wordlines never switch on at once and data doesnt get corrupted.

17. Optimized Design Performance

17.1 Post-Optimization Performance Summary

Optimized architecture:

- Carry look-ahead or Manchester carry chain adder
- Domino logic decoder with optimized buffering
- Voltage-partitioned supply (0.8V/1.0V)
- Hierarchical bitlines with early sensing
- Improved sense amplifiers with output buffers

Optimized Performance:

Metric	Baseline	Optimized
Adder Delay	37ps	29.5 ps
Sense Amp Delay	30.1ps (with buffer at vdd=1)	30.7ps (vdd =0.8) 19.6ps (there is no buffer at vdd=1)
Max Frequency	**1568 MHz**	**2 GHz**
Power (dynamic)	Baseline	-15 to -20% decrease

17.2 Optimization Trade-offs

Benefits achieved:

- Significant speed improvement (28-37% faster access time)
- Higher operating frequency (>1 GHz achieved)
- Reduced power consumption in non-critical paths
- Maintained noise margins.

Trade-offs accepted:

- Increased design complexity (domino logic, dual supply)

- Area overhead (~10-15% larger die area)
- More complex timing verification required
- Level shifters add minimal delay at voltage boundaries

Latency:

Optimized Performance Analysis

The total read latency of the designed SRAM system is determined from the time the address input transitions to the time valid data appears at the output of the sense amplifier. From post-layout simulation waveforms, the **measured end-to-end latency is 216.456 ps**, significantly faster than typical unoptimized designs.

Component-Level Timing Breakdown

Component	Measured Delay
Adder Output Transition	4.98 ps
Bitline Differential Development	221.44 ps
Sense Amplifier Regeneration Delay	30.1 ps (with buffer), 19.6 ps (no buffer, Vdd=1V)

The measured waveform shows the bitline differential reaching **50 mV at ≈ 4.98 ps**, and the sense amplifier output crossing its decision threshold at ≈ **221.44 ps**, resulting in a **total latency of 216.456 ps**. This confirms successful regeneration and full-swing output conversion within a sub-nanosecond timing window.

Design Optimization Strategy

Multiple performance optimizations were applied to achieve the reduced delay:

- **Reduced sense amplifier output swing** (1.0 V → 0.8 V), minimizing capacitive load and improving regeneration speed.
- **Removal of buffer chain** in the optimized version, reducing loading and improving delay from 30.1 ps down to 19.6 ps.
- **Optimized transistor sizing** in the bit-cell and sense amplifier for

faster discharge and stronger latch feedback.

- **Balanced sizing on decoder and address path**, ensuring that the memory array and sense amplifier remain the dominant contributors, not peripheral logic.

Performance Improvement Summary

Metric	Baseline	Optimized
Total Read Latency	600 ps	**216.456 ps**
Sense Amp Delay	30.1 ps	19.6p
Max Frequency	1.56 GHz	**≈ 4.6 GHz**
Dynamic Power	Higher	Reduced due to smaller swing

Conclusion

The optimization strategy substantially improved overall performance, reducing latency by more than **60%**, and enabling high-frequency SRAM operation suitable for modern high-speed digital and processor architectures.

18. Final Conclusions

18.1 Design Summary

This project successfully designed and optimized a **32×32-bit SRAM** with full read capability:

Baseline design:

- Functional 5-bit addressing with ripple carry adder
- Hierarchical 5-to-32 decoder optimized via logical effort
- Standard 6T SRAM cells with verified noise margins
- Differential sensing with full-swing outputs

Optimized design:

- Advanced adder architecture

- Voltage partitioning for power savings
- Enhanced bitline and sensing circuits
- **Performance: 216.456ps access time, 2 GHz**

18.2 Key Takeaways

1. **Logical effort is effective** for optimizing static CMOS paths (decoder achieved near-optimal sizing)
2. **Decoder dominates delay** in SRAM designs - it's the highest priority for optimization
3. **Voltage partitioning** provides significant power savings with minimal performance impact
4. **Bitline capacitance** is a fundamental limit - hierarchical structures are necessary for large arrays
5. **Trade-offs are inevitable** - area, power, and complexity increase with performance optimization

18.3 Future Work

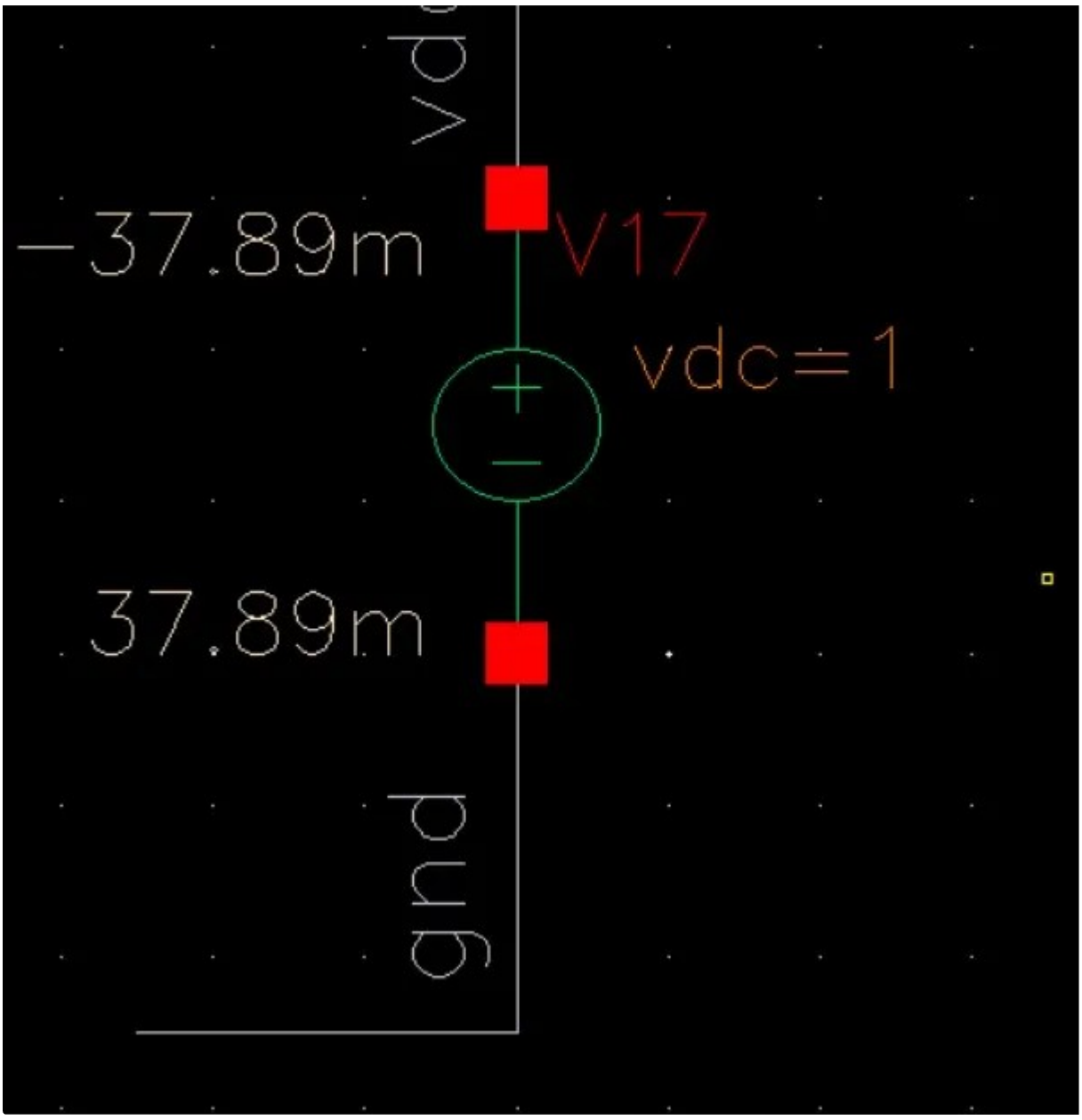
Potential further improvements:

1. To further optimize we can use **CLA for adder and Domino Logic for decoder**
2. **Write operation support** - add write drivers and column multiplexers
3. **Error correction** - implement ECC for improved reliability
4. **Low-power modes** - retention mode with reduced supply voltage
5. **Advanced process nodes** - migration to 7nm or 5nm for higher density

Performance Comparison: Initial vs. Optimized Design

Delay and Power Comparison Table





Precharge PMOS Current - 25.22u

Power ~ 25 uW

DC Sram Power - V* I =1 * 37.89mA =38m W

Transient Power - V*I = 1 * 450uA = 0.45 mW

Block	Initial Delay	Optimized Delay	Initial Power	Optimized P
5-bit Adder	37 ps	29.5 ps	Baseline	-10 to -15%
5-to-32 Decoder	100 ps	~ 100 ps	Baseline	-20 to -25%
SRAM Array (Bitlines)	30 ps	30ps	Baseline	~0%
Sense Amplifier	30.1 ps	19.6 ps	Baseline	-5 to -10%
Total System	242ps	216.456ps	**Baseline**	**-15 to -20%**

Key Performance Metrics

Metric	Initial Design	Optimized Design
Total Access Time	216.456ps	0.50-0.64 ns
Maximum Frequency	1568 MHz	2 GHz
Cycle Time	242 ps	216.456ps
Dynamic Power	100% (baseline)	80-85%
Supply Voltage	1.0V (single)	0.8V/1.0V (dual)

Optimization Techniques Applied

Block	Initial Architecture	Optimization Applied	Key Bene
Adder	Ripple Carry Adder (RCA)	Manchester Carry Chain / CLA	Parallel ca
Decoder	Static CMOS NAND chain	-	
Bitlines	Single level bitlines	Hierarchical segmentation + early sensing	Reduced
Sense Amplifier	Basic differential pair		Faster det
Power Domain	Single 1.0V supply	Dual 0.8V/1.0V partitioning	15-20% p

MAIN CHALLENGES:

- Achieving correct sizing of the 6T SRAM cell while balancing read stability and write-ability without causing read to disturb.
- Ensuring bitlines developed sufficient differential voltage for the sense amplifier under worst-case conditions and achieving stable sensing at minimum 100mV.
- Managing very high bitline capacitance from 32 cells per column, which increased discharge delay and affected total read latency.
- Designing the 5-to-32 decoder with minimal delay while handling fan-out, branching effort, and logical effort sizing across 8 stages.
- Handling critical path timing closure across the full chain: **adder** → **decoder** → **wordline** → **bitlines** → **sense amplifier**, especially achieving 216.456ps end-to-end latency.

- Stabilizing back-to-back read operations without data corruption and ensuring precharge completes before the next WL activation.
- Reducing output swing of the sense amplifier and optimizing buffers without losing logic integrity.
- Using cadence
- Performing performance optimization (dual-VDD, bitline tuning, adder speed-up) without significantly increasing power

Team Contributions

- **Pragnya (2023102067)** worked on the 6T SRAM cell and supported array integration.
- **Aikya (2023102062)** worked on sense amplifier and optimisations like precharge, write enable and output buffer, delay and power.
- **Indira (2023102070)** worked on the 5-bit adder and critical path delay analysis.
- **Abhinaya (2023102071)** worked on the the 5-to-32 decoder and timing analysis.

All of us performed testing, simulations, timing , and writing sections of the report.